

Chapter 13

Data Structures: Standard Library Containers and Iterators

Learning Objectives

- Learn more about the C++ standard library's reusable containers, iterators and algorithms.
- Understand how containers relate to C++20 ranges.
- Use I/O stream iterators to read values from the standard input stream and write values to the standard output stream.
- Use iterators to access container elements.
- Use the vector, list and deque sequence containers.
- Use ostream iterators with the `std::copy` and `std::ranges::copy` algorithms to output container elements in a single statement.
- Use the set, multiset, map and multimap ordered associative containers.
- Understand the differences between the ordered and unordered associative containers.
- Use the stack, queue and priority_queue container adaptors.
- Use the bitset “near container” to manipulate a collection of bit flags.

Introduction

- The standard library defines powerful, template-based, reusable components that implement many common data structures and algorithms used to process those data structures.
- We began introducing templates in Chapters 5–6 and use them extensively here and in Chapters 14 and 15.
- Historically, the features presented in this chapter were referred to as the **Standard Template Library** or **STL**.
 - In the C++ standard document, they are simply referred to as part of the C++ standard library.

Introduction (Cont.)

- Containers, Iterators and Algorithms
 - This chapter introduces three key standard library components—containers (templated data structures), iterators and algorithms.
 - We'll introduce **containers**, **container adaptors** and **near containers**.
- Common Member Functions Among Containers
 - Each container has associated member functions—a subset of these is defined in all containers.
 - We illustrate most of this common functionality in our examples of **array**, **vector**, **list** and **deque**.

Introduction (Cont.)

- Iterators

- Iterators, which have properties similar to pointers, are **used to manipulate container elements**.
 - ◆ Built-in arrays also can be manipulated by standard library algorithms using pointers as iterators.
- We'll see that manipulating containers via iterators provides tremendous expressive power when combined with standard library algorithms—sometimes reducing many lines of code to a single statement.

Introduction (Cont.)

- Algorithms

- Standard library algorithms (which we'll cover in Chapter 14) are function templates that perform common data manipulations, such as searching, sorting, copying, transforming and comparing elements or entire containers.
- The standard library provides over 100 algorithms:
 - ◆ 90 in the `<algorithm>` header's `std` namespace—82 also are overloaded in the `std::ranges` namespace for use with C++20 ranges,
 - ◆ 11 in the `<numeric>` header's `std` namespace,
 - ◆ 14 in the `<memory>` header's `std` namespace—all 14 also are overloaded in the `std::ranges` namespace for use with C++20 ranges and
 - ◆ 2 in the `<cstdlib>` header.
- For the complete list of algorithms with links to their descriptions, visit:
<https://en.cppreference.com/w/cpp/algorithm>

Introduction (Cont.)

- Algorithms

- Most algorithms use iterators to access container elements.
- Each algorithm has minimum requirements for the kinds of iterators that can be used with it.
 - ◆ We'll see that containers support specific kinds of iterators, some more powerful than others.
- The iterators a container supports determine whether the container can be used with a specific algorithm.
- **Iterators encapsulate the mechanisms used to traverse containers and access their elements**, enabling many algorithms to be applied to containers with significantly different implementations.
- This also enables you to create new algorithms that can process the elements of multiple container types.

Introduction (Cont.)

- C++20 Ranges

- Chapter 6 introduced C++20's ranges and views.
- You saw that a range is a collection of elements you can iterate over. So, arrays and vectors are ranges.
- You also used views to specify pipelines of operations that manipulate ranges of elements.
- Any container with iterators representing its beginning and end can be treated as a C++20 range.
- In this chapter, we'll use the C++20 standard library algorithm `std::ranges::copy` and the older C++ standard library algorithm `std::copy` to demonstrate how ranges simplify your code.
- In Chapter 14, we'll use many more C++20 algorithms from the `std::ranges` namespace to demonstrate additional ranges and views features.

Introduction (Cont.)

- Custom Templatized Data Structures

- Some popular data structures include linked lists, queues, stacks and binary trees:
 - ◆ **Linked lists** are collections of data items logically “lined up in a row.” Insertions and removals are made anywhere in a linked list.
 - ◆ **Stacks** are important in compilers and operating systems. Insertions and removals are made only at one end of a stack—its top.
 - ◆ **Queues** represent waiting lines. Insertions are made at the back (also called the tail), and removals are made from the front (also called the head).
 - ◆ **Binary trees** are nonlinear, hierarchical data structures that facilitate searching and sorting data and duplicate elimination.

Introduction (Cont.)

- Custom Templated Data Structures

- Each of these data structures has many other interesting applications.
- We can carefully weave linked objects together with pointers, **but pointer-based code is complex and can be error-prone.**
 - ◆ The slightest omissions or oversights can lead to serious memory-access violations and memory leaks with no forewarning from the compiler.
- Avoid reinventing the wheel. When possible, use the C++ standard library's preexisting containers, iterators and algorithms.
 - ◆ The prepackaged container classes provide the data structures you need for most applications.
 - ◆ Using the standard library's proven containers, iterators and algorithms helps you reduce testing and debugging time: They were conceived and designed for performance and flexibility.

Introduction.

- Custom Templatized Data Structures
 - This chapter begins with two special computer science sections on
 - ◆ **Big O notation** for expressing how hard algorithms have to work based on the number of items they process, and
 - ◆ **hash tables** for high-speed storage and retrieval of data in applications with common real-world characteristics.

A Brief Intro to Big O

- In this section, we'll discuss what Big O means and explain the significance of several popular Big O notations, including $O(1)$, $O(n)$, $O(\log n)$, and $O(n^2)$.
- On today's desktop computers, which commonly perform billions of operations per second, an algorithm's efficiency, as expressed by its Big O , translates to whether a program will run almost instantaneously or take seconds, minutes, hours, days, months or even years to complete.
- Obviously, you'd prefer algorithms that complete quickly, even though the number of items, n , they may be processing is large.
- This is especially true in today's world of Big Data computing applications.
- We'll mention Big O briefly here, use it extensively in Chapter 14 to characterize the efficiency of standard library algorithms and use it in Chapter 21 to characterize the efficiency of various popular searching and sorting algorithms we'll implement.

A Brief Intro to Big O (Cont.)

- $O(1)$ algorithms

- Suppose an algorithm tests whether the first element of an array is equal to the second.
 - ◆ If the array has 10 elements, this algorithm requires one comparison.
 - ◆ If the array has 1,000 elements, it still requires one comparison.
- In fact, the algorithm is independent of the number, n , of elements in the array.
- This algorithm is said to have **constant running time**, which is represented in Big O notation **as $O(1)$** and pronounced as “order one.”
 - ◆ An algorithm that’s $O(1)$ does not necessarily require only one comparison.
 - ◆ $O(1)$ just means that the number of comparisons is constant—it **does not grow as the size of the array increases**. An algorithm that tests whether the first element of an array is equal to any of the next three elements is still $O(1)$ even though it requires three comparisons.

A Brief Intro to Big O (Cont.)

- $O(n)$ algorithms

- An algorithm that tests whether the first array element is equal to any of the other array elements will require at most $n - 1$ comparisons, where n is the number of array elements.
 - ◆ If the array has 10 elements, this algorithm requires up to nine comparisons.
 - ◆ If the array has 1,000 elements, it requires up to 999 comparisons.
- As n goes larger, the n part of the expression $n - 1$ “dominates,” and subtracting one becomes inconsequential.
- **Big O is designed to highlight these dominant terms and ignore terms that become unimportant as n grows.**
 - ◆ For this reason, an algorithm that requires a total of $n - 1$ comparisons (such as the one we described earlier) is said to be $O(n)$.
 - ◆ An $O(n)$ algorithm is referred to as having a linear running time. $O(n)$ is pronounced “on the order of n ” or simply “order n .”

A Brief Intro to Big O (Cont.)

- $O(n^2)$ algorithms

- Now suppose you have an algorithm that tests whether any element of an array is duplicated elsewhere in the array.
 - ◆ The first element must be compared with all other elements in the array.
 - ◆ The second element must be compared with all elements except the first (it was already compared to the first).
 - ◆ The third element must be compared with all elements except the first two.
 - ◆ In the end, this algorithm makes $(n - 1) + (n - 2) + \dots + 2 + 1 = n(n - 1)/2$ comparisons.
 - ◆ As n increases, the n^2 term dominates, and the n term becomes inconsequential.

A Brief Intro to Big O (Cont.)

- $O(n^2)$ algorithms

- Big O is concerned with how an algorithm's running time grows in relation to the number of items processed.
- Suppose an algorithm requires n^2 comparisons.
 - ◆ With four elements, the algorithm requires 16 comparisons; with eight elements, 64 comparisons.
 - ◆ With this algorithm, doubling the number of elements quadruples the number of comparisons.
- Consider a similar algorithm requiring $n^2/2$ comparisons.
 - ◆ With four elements, the algorithm requires eight comparisons; with eight elements, 32 comparisons.
 - ◆ Again, doubling the number of elements quadruples the number of comparisons.
- Both of these algorithms grow as the square of n , so **Big O ignores the constant factor**, and both algorithms are considered to be $O(n^2)$, which is referred to as quadratic running time and pronounced “on the order of n-squared” or simply “order n-squared.”

A Brief Intro to Big O (Cont.)

- Efficiency of the Linear Search $O(n)$
 - The linear search algorithm, which is typically used for searching an unsorted array, runs in $O(n)$ time.
 - ◆ The worst case in this algorithm is that every element must be checked to determine whether the search key exists in the array.
 - ◆ If array's size doubles, the number of comparisons that the algorithm must perform also doubles.
 - Linear search can provide outstanding performance if the element matching the search key is at or near the front of the array.
 - But we seek algorithms that perform well, on average, across all searches, including those where the element matching the search key is at or near the end of the array.

A Brief Intro to Big O (Cont.)

- Efficiency of the Linear Search $O(n)$
 - Linear search is easy to program, but it can be slow compared to other search algorithms, especially as n gets large.
 - If a program needs to perform many searches on large arrays, it's better to use a more efficient algorithm, such as the C++ standard library algorithm `binary_search`, which we used in Chapter 6 and will use again in Chapter 14.
 - ◆ We'll show how to implement the binary search algorithm in Chapter 21.
 - **Sometimes the simplest algorithms perform poorly.**
 - ◆ Their virtue often is that they're easy to program, test and debug.
 - ◆ Sometimes more complex algorithms are required to realize maximum performance.

A Brief Intro to Big O (Cont.)

- Efficiency of the Binary Search $O(\log n)$
 - In the worst-case scenario, searching a sorted array of 1,023 elements ($2^{10} - 1$) takes only 10 comparisons with binary search.
 - The algorithm compares the search key to the middle element of the sorted array.
 - ◆ If it's a match, the search is over. More likely, the search key will be larger or smaller than the middle element:
 - ◆ If it's larger, we can eliminate the array's first half from further consideration.
 - ◆ If it's smaller, we can eliminate the array's second half from further consideration.
 - This creates a “halving effect” so that on subsequent searches, we have to search only 511, 255, 127, 63, 31, 15, 7, 3 and 1 elements. The number 1,023 ($2^{10} - 1$) needs to be halved only 10 times to either find the key or determine that it's not in the array.
 - ◆ Dividing by 2 is equivalent to one comparison in the binary search algorithm.

A Brief Intro to Big O (Cont.)

- Efficiency of the Binary Search $O(\log n)$
 - Thus, a much larger array of 1,048,575 ($2^{20} - 1$) elements takes a maximum of 20 comparisons to find the key, and an array of about one billion elements takes a maximum of 30 comparisons to find the key.
 - This is a tremendous improvement in performance over the linear search.
 - ◆ For a one billion-element array, this is a difference between an average of 500 million comparisons for the linear search and a maximum of only 30 comparisons for the binary search!
 - ◆ The maximum number of comparisons needed for the binary search of any sorted array is the exponent of the first power of 2 greater than the number of elements in the array, which is represented as $\lg n$.
 - All logarithms grow at “roughly the same rate,” so for Big O comparison purposes, **the base can be omitted**. This results in a Big O of $O(\log n)$ for a binary search, which is also known as **logarithmic running time** and pronounced as “order log n .”

A Brief Intro to Big O (Cont.)

- Comparing Big O Notations

- The following table lists various common Big O notations and several values for n to highlight the differences in the growth rates. If you interpret the values in the table as seconds of calculation, you can easily see why $O(n^2)$ algorithms are to be avoided!

$n =$	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$
1	1	0	1	0	1
2	1	1	2	2	4
3	1	1	3	3	9
4	1	1	4	4	16
5	1	1	5	5	25
10	1	1	10	10	100
100	1	2	100	200	10,000
1,000	1	3	1,000	3,000	10^6
1,000,000	1	6	1,000,000	6,000,000	10^{12}

A Brief Intro to Big O .

- Comparing Big O Notations

- In the implementation of the standard library containers (Chapter 13) and algorithms (Chapter 14), Big O s of $O(1)$, $O(n)$, $O(\log n)$ and even $O(n \log n)$ —which is common for relatively good sorting algorithms—are considered to be reasonably efficient.
- Algorithms categorized by $O(n^2)$ or worse, such as $O(2^n)$ or $O(n!)$, could run on a modest number of items for centuries, millennia or longer.
- So you'll want to avoid writing such algorithms.

A Brief Intro to Hash Tables

- When a program creates objects, it may need to store and retrieve them efficiently.
- Storing and retrieving information with arrays is efficient if some aspect of your data directly matches a numerical key value and if the keys are unique and tightly packed.
- If you have 100 employees with nine-digit Social Security numbers and you want to store and retrieve employee data by using the Social Security number as an array index, the task will require an array with over 800 million elements because nine-digit Social Security numbers must begin with 001–899 (excluding 666) as per the Social Security Administration's website:
<https://www.ssa.gov/employer/randomization.html>
- A program having so large an array could achieve high performance for both storing and retrieving employee records by simply using the Social Security number as the array index.
 - This is impractical for most applications that use Social Security numbers as keys.

A Brief Intro to Hash Tables (Cont.)

- Numerous applications have this problem—namely, that either
 - the keys are of the wrong type (e.g., not positive integers usable as array subscripts) or
 - they're of the right type, but sparsely spread over a huge range, such as Social Security numbers for a small company's employees.
- What is needed is a high-speed scheme for converting keys—such as Social Security numbers, inventory part numbers and the like—into unique array indices over a modest-size array.
 - Then, when an application needs to store a data item, the scheme can convert the application's key rapidly into an array index (a process called hashing), and the item can be stored at that slot in the array.
 - Retrieval is accomplished the same way: Once the application has a key for which it wants to retrieve the data, it simply applies the conversion to the key (again, called hashing), producing the array index where the data is stored and retrieved.

A Brief Intro to Hash Tables (Cont.)

- Why the name hashing?
 - When we convert a key into an array index, we literally scramble the bits, forming a kind of “mishmashed,” or hashed, number.
 - The number actually has no real significance beyond its usefulness in storing and retrieving particular data.

A Brief Intro to Hash Tables (Cont.)

- Collisions

- A glitch in the scheme is called a collision, which occurs when two different keys “hash into” the same array element (or cell).
- We cannot store two values in the same space, so we need to find an alternative home for all values beyond the first that hash to the same array index.
- There are many schemes for doing this. One is to “hash again”—that is, to apply another hashing transformation to the previous hash result to provide the next candidate cell in the array.
- The hashing process is designed to distribute the values throughout the table, so hopefully an available cell will be found with one or a few hashes.

A Brief Intro to Hash Tables (Cont.)

- Collisions

- Another scheme uses one hash to locate the first candidate cell.
- If that cell is occupied, adjacent cells are searched sequentially until an available cell is found.
- Retrieval works the same way: The key is hashed once to determine the initial location and check whether it contains the desired data.
 - ◆ If it does, the search is finished;
 - ◆ otherwise, successive cells are searched sequentially until the desired data is found.
- A popular solution to hash-table collisions is to have each cell of the table be a **hash bucket**—typically a **list of all the key–value pairs that hash to that cell**.

A Brief Intro to Hash Tables (Cont.)

- Load Factor

- A hash table's load factor affects the performance of hashing schemes.
- The load factor is **the ratio of the hash table's number of occupied to its total number of cells**. The closer this ratio gets to 1.0, the greater the chance of collisions, which slow data insertion and retrieval.
- A hash table's load factor is a classic example of a **memory-space/execution-time trade-off**:
 - ◆ By increasing the load factor, we get better memory utilization, but the program runs slower due to increased hashing collisions.
 - ◆ By decreasing the load factor, we get better program speed, because of reduced hashing collisions, but we get poorer memory utilization because a larger portion of the hash table remains empty.

A Brief Intro to Hash Tables (Cont.)

- Containers Based on Hash Tables

- C++'s [associative containers](#) `unordered_set`, `unordered_multiset`, `unordered_map` and `unordered_multimap`, which we'll study in this chapter, are implemented as hash tables “under the hood.”
- When you use those containers, you get the benefit of high-speed data storage and retrieval without having to build your own hash-table mechanisms—a classic example of software reuse.

A Brief Intro to Hash Tables.

- Hashing and Operating Systems

- Another interesting application of hashing is with virtual memory operating systems.
- Programs and data are maintained on massive secondary storage and brought into more limited primary memory (and even more limited and faster cache memory) to execute.
- The trick to running applications efficiently is to keep in primary memory (and cache memory) the portion of the program that needs to execute at a given time.
 - ◆ Finding those program pieces in virtual memory involves searching the very large data structures that define it.
 - ◆ Hashing is often used for that purpose.

Introduction to Containers

- The standard library containers are divided into four major categories:
 - sequence containers,
 - ordered associative containers,
 - unordered associative containers and
 - container adaptors.
- We briefly summarize the containers here and show many live-code examples later.

Introduction to Containers (Cont.)

- Sequence Containers

- The sequence containers represent linear data structures with all of their elements conceptually “lined up in a row,” such as arrays, vectors and linked lists, all of which are sortable.

Introduction to Containers (Cont.)

- The Five Sequence Containers

- Array

- ◆ Fixed size.
 - ◆ Elements are contiguous in memory.
 - ◆ Direct access (also called random access) to any element.

- Vector

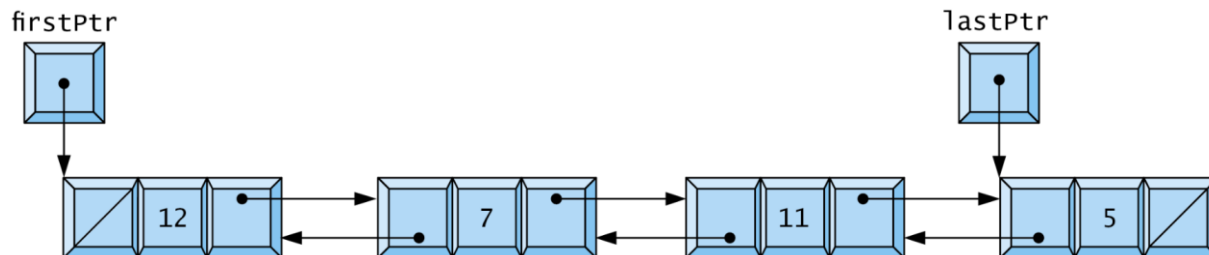
- ◆ Resizable.
 - ◆ Elements are contiguous in memory.
 - ◆ Rapid insertions and deletions at the back.
 - ◆ Direct access to any element.

Introduction to Containers (Cont.)

- The Five Sequence Containers

- list

- ◆ Resizable doubly linked list, rapid insertion and deletion anywhere.
 - ◆ A doubly linked list supports iterating forward or backward through the list and is often implemented with two “start pointers”: one points to the list’s first element and one to the last.
 - ◆ Each node contains pointers to the previous node and next node (hence the term doubly linked list), as illustrated by the horizontal arrows in the following diagram.
 - ◆ The slashes (/) in the first and last nodes represent nullptrs.

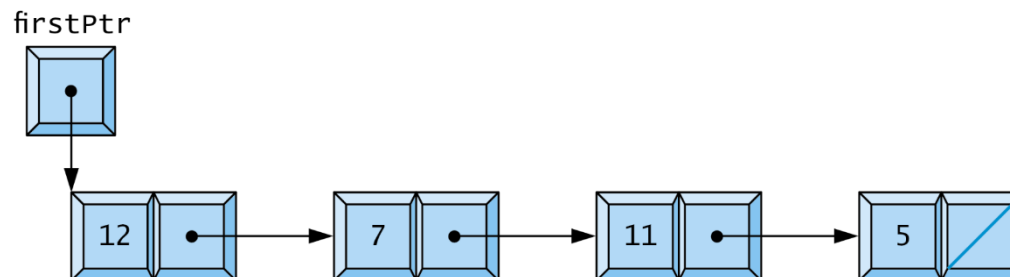


Introduction to Containers (Cont.)

- The Five Sequence Containers

- forward_list

- ◆ Resizable singly linked list, rapid insertion and deletion anywhere.
 - ◆ A singly linked list typically maintains a pointer to its first node, and
 - ◆ each node contains one pointer to the next node (hence the term singly linked list), as illustrated by the horizontal arrows in the following diagram.
 - ◆ The slash (/) in the last node represents a nullptr:



Introduction to Containers (Cont.)

- Sequence Containers

- Linked-list containers do not support random access to any element.
- Class string supports the same functionality as a sequence container but stores only character data.

Introduction to Containers (Cont.)

- Associative Containers

- Associative containers are nonlinear data structures (usually implemented as binary trees) that typically can quickly locate elements.
- Such containers can store sets of values or key–value pairs in which each key has an associated value.
 - ◆ For example, a program might associate employee IDs with Employee objects.
 - ◆ As you'll see, some associative containers allow multiple values for each key.
 - ◆ The keys in associative containers are **immutable**—they cannot be modified unless you first remove them from the container.

Introduction to Containers (Cont.)

- Associative Containers

- The four ordered associative containers are

- ◆ multiset

- Rapid lookup,
 - duplicates allowed.

- ◆ Set

- Rapid lookup,
 - no duplicates allowed.

- ◆ Multimap

- Rapid key-based lookup,
 - duplicate keys allowed.
 - One-to-many mapping.

- ◆ Map

- Rapid key-based lookup,
 - no duplicate keys allowed.
 - One-to-one mapping.

Introduction to Containers (Cont.)

- Associative Containers

- The four unordered associative containers (implemented using hashing) are
 - ◆ `unordered_multiset`
 - ◆ `unordered_set`
 - ◆ `unordered_multimap`
 - ◆ `unordered_map`
- Their characteristics are the same as their ordered versions.

Introduction to Containers (Cont.)

- Container Adaptors

- The standard library implements `stack`, `queue` and `priority_queue` as container adaptors that enable a program to view a sequence container in a constrained manner.
- The three container adaptors are
 - ◆ `stack`—Last-in, first-out (LIFO) data structure.
 - ◆ `queue`—First-in, first-out (FIFO) data structure.
 - ◆ `priority_queue`—Highest-priority element is always the first element out.

Introduction to Containers (Cont.)

- Near Containers

- There are other container types that are considered near containers—**built-in arrays**, **bitsets** for maintaining sets of flag values, **strings** and **valarrays** for performing high-speed mathematical vector operations (not to be confused with the vector container).
- These types are considered near containers because they exhibit some, but not all, capabilities of the sequence and associative containers.

Introduction to Containers (Cont.)

- Common Nested Types in Sequence and Associative Containers
 - The table below shows the common types defined inside each sequence and associative container's class definition.
 - These nested types are used in template-based variable declarations, function parameters and function return types, as you'll see in this chapter and Chapter 14.
 - For example, each container's `value_type` always represents the container's element type.

Introduction to Containers (Cont.)

- Common Nested Types in Sequence and Associative Containers

Nested type	Description
<code>allocator_type</code>	The type of object used to allocate the container's memory —not included in the array container. Containers that use allocators each provide a default allocator, which is sufficient for most programmers. Custom allocators are beyond this book's scope.
<code>value_type</code>	The type of the container's elements.
<code>reference</code>	The type used to declare a reference to a container element.
<code>const_reference</code>	The type used to declare a reference to a <code>const</code> container element.
<code>pointer</code>	The type used to declare a pointer to a container element.
<code>const_pointer</code>	The type used to declare a pointer to a <code>const</code> container element.

Introduction to Containers (Cont.)

- Common Nested Types in Sequence and Associative Containers

iterator	An iterator that points to a container element.
const_iterator	An iterator that points to an element of a const container. Used only to read elements and to perform const operations.
reverse_iterator	A reverse iterator that points to a container element. Iterates through a container back-to-front . Not provided for forward_list .
const_reverse_iterator	A reverse iterator that points to a container element and can be used only to read elements and to perform const operations . Used to iterate through a container in reverse. Not provided for forward_list .
difference_type	A signed type representing the number of elements between two iterators that refer to elements of the same container. A value of this type is returned by an iterator's overloaded minus (-) operator and by the function <code>std::distance</code> .
size_type	The unsigned type used to count items in a container and index through a random-access sequence container.

Introduction to Containers (Cont.)

- Common Container Member and Non-Member Functions
 - Most containers provide similar functionality.
 - ◆ Many operations apply to all containers, and others apply to specific container categories.
 - ◆ Before using any container, you should study its capabilities.
 - The following tables describe the commonly available functions in most standard library containers.
 - For a complete list of all the container member functions and which containers support them, see the [cppreference.com](https://en.cppreference.com/w/cpp/container) member function table:
<https://en.cppreference.com/w/cpp/container>
 - Several member functions we do not list in this section are covered throughout the chapter as we present various sequence containers, associative containers and container adaptors.

Introduction to Containers (Cont.)

- Container Special Member Functions

- The following table describes the special member functions provided by each container.
- In addition, each container class typically provides many overloaded constructors for initializing containers and container adaptors in various ways.
 - ◆ For example, each sequence and associative container can be initialized from an `initializer_list`.

Introduction to Containers (Cont.)

- Container Special Member Functions

Container special member function	Description
default constructor	A constructor that initializes an empty container.
copy constructor	A constructor that initializes the container to be a copy of an existing container of the same type.
copy operator=	Copies the elements of one container into another.
move constructor	Moves the contents of an existing container into a new one of the same type. The old container no longer contains the data. This avoids the overhead of copying each element of the existing container.
move operator=	Moves the contents of one container into another of the same type. The old container no longer contains the data. This avoids the overhead of copying each element of the existing container.
destructor	Destructor function to destroy the container elements.

Introduction to Containers (Cont.)

- Non-Member Relational and Equality Operators
 - The `<`, `<=`, `>`, `>=`, `==` and `!=` operators are supported by most containers.
 - ◆ In C++17 and earlier, these are overloaded as non-member functions.
 - ◆ In C++20, the `<`, `<=`, `>`, `>=` and `!=` operators are synthesized by the compiler using the new three-way comparison operator `<=>` and the `==` operator.
 - As we showed in Chapter 11, the C++ compiler can use `<=>` to implement each relational and equality comparison by rewriting it in terms of `<=>`.
 - ◆ For example, the compiler rewrites `x < y` as `(x <=> y) < 0`
 - The unordered associative containers do not support `<`, `<=`, `>` and `>=`.
 - The relational and equality operators are not supported for `priority_queues`.

Introduction to Containers (Cont.)

- Member Functions That Return Iterators
 - The following table shows container member functions that return iterators.
 - ◆ The container `forward_list` does not have the member functions `rbegin`, `rend`, `crbegin` and `crend`.

Introduction to Containers (Cont.)

- Member Functions That Return Iterators

<code>begin</code>	Returns an iterator or a <code>const_iterator</code> (depending on whether the container is <code>const</code>) referring to the container's first element .
<code>end</code>	Returns an iterator or a <code>const_iterator</code> (depending on whether the container is <code>const</code>) referring to the next position after the end of the container.
<code>cbegin</code>	Returns a <code>const_iterator</code> referring to the container's first element .
<code>cend</code>	Returns a <code>const_iterator</code> referring to the next position after the end of the container.
<code>rbegin</code>	Returns a <code>reverse_iterator</code> or a <code>const_reverse_iterator</code> (depending on whether the container is <code>const</code>) referring to the container's last element .
<code>rend</code>	Returns a <code>reverse_iterator</code> or a <code>const_reverse_iterator</code> (depending on whether the container is <code>const</code>) referring to the position before the container's first element .
<code>crbegin</code>	Returns a <code>const_reverse_iterator</code> referring to the container's last element .
<code>crend</code>	Returns a <code>const_reverse_iterator</code> referring to the position before the container's first element .

Introduction to Containers (Cont.)

- Requirements for Container Elements

- Before using a standard library container, it's important to ensure that the element type supports a minimum set of functionality. For example:
 - ◆ If inserting an item into a container requires a copy of the object, the object type should provide a copy constructor and copy assignment operator.
 - ◆ If inserting an item into a container requires moving the object, the object type should provide a move constructor and move assignment operator.
 - ◆ The ordered associative containers and many algorithms require elements to be compared. For this reason, the object type should support comparisons.

Introduction to Containers.

- Requirements for Container Elements

- As you review the documentation for each container, whether in the C++ standard document itself or on sites like cppreference.com, you'll see various named requirements, such as CopyConstructible, MoveAssignable or EqualityComparable.
- These help you determine whether your types are compatible with various C++ standard library containers and algorithms.
- You can view a list of named requirements and their descriptions at https://en.cppreference.com/w/cpp/named_req
- In C++20, many named requirements are formalized as concepts, which we'll discuss in Chapters 14 and 15.

Working with Iterators

- Iterators have many similarities to pointers.
 - They point to elements in sequence containers and associative containers,
 - but **they also hold state information sensitive to the particular containers on which they operate.**
- So, iterators are implemented for each type of container.
- Iterator operation syntax is uniform across containers.
 - For example, applying `*` to an iterator dereferences it so that you can access the referenced element.
 - Applying `++` to an iterator moves it to the container's next element.
- This uniformity is a key aspect of iterators, enabling standard library algorithms to operate on various container types using compile-time polymorphism.

Working with Iterators (Cont.)

- Sequence containers and associative containers provide member functions `begin` and `end`.
 - Function `begin` returns an iterator pointing to the container's first element.
 - Function `end` returns an iterator pointing to **the first element past the end of the container** (one past the end)—a nonexistent element that's frequently used to determine when the end of a container is reached.
- You'll often use this in equality or inequality comparisons to determine whether an incremented iterator has reached the end of the container.

Working with Iterators (Cont.)

- Using `istream_iterator` for Input and `ostream_iterator` for Output
 - We use iterators with sequences (also called ranges). These can be in containers, or they can be input sequences or output sequences.
 - The next program demonstrates
 - ◆ `input` from the standard input (a sequence of data for program input) `using an istream_iterator`, and
 - ◆ `output` to the standard output (a sequence of data for program output) `using an ostream_iterator`.

Working with Iterators (Cont.)

- Using `istream_iterator` for Input and `ostream_iterator` for Output
 - The program inputs two integers from the user and displays their sum.
 - As you'll see later in this chapter, `istream_iterators` and `ostream_iterators` can be used with the standard library algorithms to create powerful statements.
 - ◆ For example, subsequent examples will use an `ostream_iterator` with the `copy` algorithm to copy a container's elements to the standard output stream with a single statement.

Working with Iterators (Cont.)

```
1 // main.cpp
2 // Demonstrating input and output with iterators.
3 #include <iostream>
4 #include <iterator> // ostream_iterator and istream_iterator
5
6 int main() {
7     std::cout << "Enter two integers: ";
8
9     // create istream_iterator for reading int values from cin
10    std::istream_iterator<int> inputInt{std::cin};
11
12    const int number1{*inputInt}; // read int from standard input
13    ++inputInt; // move iterator to next input value
14    const int number2{*inputInt}; // read int from standard input
15
16    // create ostream_iterator for writing int values to cout
17    std::ostream_iterator<int> outputInt{std::cout};
18
19    std::cout << "The sum is: ";
20    *outputInt = number1 + number2; // output result to cout
21    std::cout << "\n";
22 }
```

Enter two integers: 12 25
The sum is: 37

Working with Iterators (Cont.)

- `istream_iterator`

- Line 10 creates an `istream_iterator` capable of extracting (inputting) `int` values from the standard input object `cin`.
- Line 12 dereferences iterator `inputInt` to read the first integer from `cin` and initializes `number1` with the value.
 - ◆ The dereferencing operator `*` applied to `inputInt` gets the value from the stream.

```
9      // create istream_iterator for reading int values from cin
10     std::istream_iterator<int> inputInt{std::cin};
11
12     const int number1{*inputInt}; // read int from standard input
13     ++inputInt; // move iterator to next input value
14     const int number2{*inputInt}; // read int from standard input
```

Working with Iterators (Cont.)

- `istream_iterator`

- Line 13 positions `inputInt` to the next value in the input stream.
 - ◆ Either prefix or postfix increment can be used.
 - ◆ We use the prefix form for performance reasons because it does not create a temporary object.
- Line 14 inputs the next int from `inputInt` and initializes `number2` with it.

```
9      // create istream_iterator for reading int values from cin
10     std::istream_iterator<int> inputInt{std::cin};
11
12     const int number1{*inputInt}; // read int from standard input
13     ++inputInt; // move iterator to next input value
14     const int number2{*inputInt}; // read int from standard input
```

Working with Iterators (Cont.)

- ostream_iterator

- Line 17 creates an ostream_iterator capable of inserting (outputting) int values in the standard output object cout.
- Line 20 outputs an integer to cout by assigning to the dereferenced iterator (*outputInt) the sum of number1 and number2.

```
16 // create ostream_iterator for writing int values to cout
17 std::ostream_iterator<int> outputInt{std::cout};
18
19 std::cout << "The sum is: ";
20 *outputInt = number1 + number2; // output result to cout
21 std::cout << "\n";
22 }
```

Enter two integers: 12 25
The sum is: 37

Working with Iterators (Cont.)

- Iterator Categories

- The following table describes the iterator categories.
- Each provides a specific set of functionality.
- Throughout this chapter, we discuss which iterator category each container supports.
- In Chapter 14, you'll see that an algorithm's minimum iterator requirements determine which containers can be used with that algorithm.

Working with Iterators (Cont.)

- Iterator Categories

Iterator category	Description
input	Used to read an element from a container. Can move only forward one element at a time from the container's beginning to its end. Input iterators support only onepass algorithms. The same input iterator cannot be used to pass through a sequence twice.
output	Used to write an element to a container. Can move only forward one element at a time. Output iterators support only one-pass algorithms. The same output iterator cannot be used to pass through a sequence twice. For the subsequent iterator types in this table, if they refer to non-constant data, the iterators can be used to write to the container.

Working with Iterators (Cont.)

- Iterator Categories

forward	Has the capabilities of input and output iterators and retains its position in the container. Such iterators can pass through a sequence more than once for multipass algorithms.
bidirectional	Has the capabilities of a forward iterator and adds the ability to move backward from the container's end toward its beginning. Bidirectional iterators support multipass algorithms.
random access	Has the capabilities of a bidirectional iterator and adds the ability to directly access any element of the container—that is, to jump forward or backward by an arbitrary number of elements. These can also be compared with relational operators.
contiguous	A random-access iterator that requires elements to be stored in contiguous memory locations.

Working with Iterators (Cont.)

- Container Support for Iterators
 - The iterator category that each container supports determines whether that container can be used with specific algorithms.
 - **Containers that support random-access iterators can be used with all standard library algorithms.**
 - Pointers into built-in arrays can be used as iterators.
 - The following table shows the iterator category of each container.
 - Sequence containers, associative containers, strings and built-in arrays are all traversable with iterators.

Working with Iterators (Cont.)

- Container Support for Iterators

Container	Iterator type	Container	Iterator type
Sequence containers		Ordered associative containers	
vector	contiguous	set	bidirectional
array	contiguous	multiset	bidirectional
deque	random access	map	bidirectional
list	bidirectional	multimap	bidirectional
forward_list	forward		
Unordered associative containers		Container adaptors	
unordered_set	forward	stack	none
unordered_multiset	forward	queue	none
unordered_map	forward	priority_queue	none
unordered_multimap	forward		

Working with Iterators (Cont.)

- Predefined Iterator Type Names

- The following table shows the predefined iterator type names found in the standard library container class definitions.

Predefined iterator type name	Direction of ++	Capability
iterator	forward	read/write
const_iterator	forward	read
reverse_iterator	backward	read/write
const_reverse_iterator	backward	read

Working with Iterators (Cont.)

- Predefined Iterator Type Names

- Not every iterator type name is defined for every container.
- The const iterators are for traversing containers that should not be modified.
- Reverse iterators traverse containers in the reverse direction.
- Operations performed on a const_iterator return references to const to prevent modifying container elements.
 - ◆ Using const_iterators where appropriate is another example of the principle of least privilege.

Working with Iterators (Cont.)

- Iterator Operators

- The following table shows operators for each iterator type.
- In addition to the operators shown for all iterators, iterators must provide default constructors (forward iterators and higher), copy constructors and copy assignment operators
- A forward iterator supports ++ and all input and output iterator capabilities.
- A bidirectional iterator supports -- and all the capabilities of forward iterators.
- Random-access iterators and contiguous iterators support all of the operations shown in the table.
- For input iterators and output iterators, it's not possible to save the iterator, then use the saved value later.

Working with Iterators (Cont.)

- Iterator Operators

Iterator operation	Description
All iterators	
<code>++p</code>	Preincrement an iterator, moving it to the next element.
<code>p++</code>	Postincrement an iterator, moving it to the next element.
<code>p = p1</code>	Assign one iterator to another.
Bidirectional iterators	
<code>--p</code>	Predecrement an iterator, moving to the previous element.
<code>p--</code>	Postdecrement an iterator, moving to the previous element.

Working with Iterators (Cont.)

- Iterator Operators

Input iterators	
<code>*p</code>	Dereference an iterator as a const lvalue .
<code>p->m</code>	Use the iterator to access the element <code>m</code> .
<code>p == p1</code>	Compare iterators for equality.
<code>p != p1</code>	Compare iterators for inequality.
Output iterators	
<code>*p</code>	Dereference an iterator as an lvalue .
<code>p = p1</code>	Assign one iterator to another.
Forward iterators	Forward iterators provide all the functionality of both input iterators and output iterators.

Working with Iterators (Cont.)

- Iterator Operators

Random-access iterators	
<code>p += i</code>	Increment the iterator <code>p</code> by <code>i</code> positions.
<code>p -= i</code>	Decrement the iterator <code>p</code> by <code>i</code> positions.
<code>p + i</code> or <code>i + p</code>	Expression value is an iterator positioned at <code>p</code> incremented by <code>i</code> positions.
<code>p - i</code>	Expression value is an iterator positioned at <code>p</code> decremented by <code>i</code> positions.
<code>p - p1</code>	Expression value is an integer representing the distance (that is, number of elements) between two elements in the same container.
<code>p[i]</code>	Return a reference to the element offset from <code>p</code> by <code>i</code> positions

Working with Iterators (Cont.)

- Iterator Operators

<code>p < p1</code>	Return true if iterator <code>p</code> is less than iterator <code>p1</code> (i.e., iterator <code>p</code> is before iterator <code>p1</code> in the container); otherwise, return false .
<code>p <= p1</code>	Return true if iterator <code>p</code> is less than or equal to iterator <code>p1</code> (i.e., iterator <code>p</code> is before iterator <code>p1</code> or at the same location as iterator <code>p1</code> in the container); otherwise, return false .
<code>p > p1</code>	Return true if iterator <code>p</code> is greater than iterator <code>p1</code> (i.e., iterator <code>p</code> is after iterator <code>p1</code> in the container); otherwise, return false .
<code>p >= p1</code>	Return true if iterator <code>p</code> is greater than or equal to iterator <code>p1</code> (i.e., iterator <code>p</code> is after iterator <code>p1</code> or at the same location as iterator <code>p1</code> in the container); otherwise, return false .

A Brief Introduction to Algorithms

- The standard library provides scores of algorithms you can use to manipulate a wide variety of containers.
 - Inserting, deleting, searching, sorting, and others are appropriate for some or all of the sequence and associative containers.
- **The algorithms operate on container elements only indirectly through iterators.**
 - Many algorithms operate on sequences of elements defined by iterators pointing to the first element of the sequence and to **one element past the last element—known as half-open ranges.**

A Brief Introduction to Algorithms.

- Many now support C++20 ranges.
 - It's also possible to create your own new algorithms that operate in a similar fashion so they can be used with the standard library containers and iterators.
- In this chapter, we'll use the copy algorithm in many examples to copy a container's contents to the standard output.
- We discuss many standard library algorithms in Chapter 14.

Sequence Containers

- The C++ standard library provides five sequence containers—array, vector, deque, list and forward_list.
 - The array, vector and deque containers are typically based on built-in arrays.
 - The list and forward_list containers implement linked-list data structures.
- We've already discussed and used array extensively in Chapter 6, so we do not cover it again here.
- We've also already introduced vector in Chapter 6—and we discuss it in more detail here.

Sequence Containers (Cont.)

- Performance and Choosing the Appropriate Container
 - We have presented the operations common to most standard library containers.
 - Beyond these operations, each container typically provides various other capabilities.
 - Many are common to several containers, but they're not always equally efficient.
 - ◆ The C++ Core Guidelines say **vector is satisfactory for most applications**.
 - ◆ However, you should profile your application to ensure the container you choose is the correct one for your use-case and performance requirements.

Sequence Containers.

- Performance and Choosing the Appropriate Container
 - **Insertion at the back of a vector is efficient.**
 - ◆ The vector grows, if necessary, to accommodate the new item.
 - ◆ It's expensive to insert (or delete) an element at the front or in the middle of a vector.
 - Every item after the insertion (or deletion) point must be moved because **vector elements occupy contiguous cells in memory.**
 - For applications that require frequent insertions and deletions at both ends of a container, use a deque rather than a vector.
 - ◆ A deque does not move any elements for insertions and deletions at the front, making it more efficient than vector.

vector Sequence Container

- The vector container is a dynamic data structure with contiguous memory locations.
- It provides rapid indexed access with the overloaded subscript operator `[]`.
- **Choose the vector container for the best random-access performance in a container that can grow.**
- When a vector's capacity is exceeded, it
 - **allocates** a larger built-in array,
 - **copies** or **moves** (depending on what the element type supports) the original elements into the new built-in array and
 - **deallocates** the old built-in array.

vector Sequence Container (Cont.)

- Using vectors and Iterators
 - The next program illustrates several vector member functions, many of which are available in every sequence container and associative container.

vector Sequence Container (Cont.)

```
1 // main.cpp
2 // Standard library vector class template.
3 #include <format>
4 #include <iostream>
5 #include <ranges>
6 #include <vector> // vector class-template definition
7
8 // display value appended to vector and updated vector size and capacity
9 void showResult(int value, size_t size, size_t capacity) {
10     std::cout << std::format("appended: {}; size: {}; capacity: {}\n",
11         value, size, capacity);
12 }
13
14 int main() {
15     std::vector<int> integers{}; // create vector of ints
16
17     std::cout << "Size of integers: " << integers.size()
18         << "\nCapacity of integers: " << integers.capacity() << "\n\n";
19
20     // append 1-10 to integers and display updated size and capacity
21     for (int i : std::views::iota(1, 11)) {
22         integers.push_back(i); // push_back is in vector, deque and list
23         showResult(i, integers.size(), integers.capacity());
24     }
25
26     std::cout << "\nOutput integers using iterators: ";
27
28     for (auto constIterator{integers.cbegin()};
29         constIterator != integers.cend(); ++constIterator) {
30         std::cout << *constIterator << ' ';
31     }
32
33     std::cout << "\nOutput integers in reverse using iterators: ";
34
35     // display vector in reverse order using const reverse iterator
36     for (auto reverselIterator{integers.crbegin()};
37         reverselIterator != integers.crend(); ++reverselIterator) {
38         std::cout << *reverselIterator << ' ';
39     }
40
41     std::cout << "\n";
42 }
```

vector Sequence Container (Cont.)

- Using vectors and Iterators

- As we add items to a `vector<int>` in this example, we call our `showResult` function (lines 9–12) to display the new value as well as the vector's
 - ◆ **size**—the number of elements it currently contains and
 - ◆ **capacity**—the number of elements it can store before it needs to **dynamically resize itself** to accommodate more elements.

```
1 // main.cpp
2 // Standard library vector class template.
3 #include <format>
4 #include <iostream>
5 #include <ranges>
6 #include <vector> // vector class-template definition
7
8 // display value appended to vector and updated vector size and capacity
9 void showResult(int value, size_t size, size_t capacity) {
10     std::cout << std::format("appended: {}; size: {}; capacity: {}\n",
11                             value, size, capacity);
12 }
```

vector Sequence Container (Cont.)

- Creating a vector and Displaying Its Initial Size and Capacity
 - Line 15 defines the `vector<int>` object `integers`. `vector`'s default constructor creates an empty vector with its size and capacity set to 0.
 - ◆ So, the vector will have to allocate memory when elements are added to it.

```
14 int main() {  
15     std::vector<int> integers{}; // create vector of ints  
16  
17     std::cout << "Size of integers: " << integers.size()  
18     << "\nCapacity of integers: " << integers.capacity() << "\n\n";  
}
```

Size of integers: 0
Capacity of integers: 0

vector Sequence Container (Cont.)

- Creating a vector and Displaying Its Initial Size and Capacity
 - Lines 17–18 display the size and capacity.
 - ◆ Function size returns the number of elements currently stored in the container.
 - ◆ Function capacity returns the vector's current capacity.

```
14 int main() {  
15     std::vector<int> integers{}; // create vector of ints  
16  
17     std::cout << "Size of integers: " << integers.size()  
18     << "\nCapacity of integers: " << integers.capacity() << "\n\n";  
}
```

```
Size of integers: 0  
Capacity of integers: 0
```

vector Sequence Container (Cont.)

```
14 int main() {
15     std::vector<int> integers{}; // create vector of ints
16
17     std::cout << "Size of integers: " << integers.size()
18         << "\nCapacity of integers: " << integers.capacity() << "\n\n";
19
20     // append 1-10 to integers and display updated size and capacity
21     for (int i : std::views::iota(1, 11)) {
22         integers.push_back(i); // push_back is in vector, deque and list
23         showResult(i, integers.size(), integers.capacity());
24     }
25
26     std::cout << "\nOutput integers using iterators: ";
27
28     for (auto constIterator{integers.cbegin()};
29         constIterator != integers.cend(); ++constIterator) {
30         std::cout << *constIterator << ' ';
31     }
32
33     std::cout << "\nOutput integers in reverse using iterators: ";
```

vector Sequence Container (Cont.)

- vector Member Function `push_back`

- Lines 21–24 call `push_back` to append one element at a time to the vector.
 - ◆ Each loop iteration calls `showResult` to display the added item and the vector's new size and capacity.
 - ◆ If the vector's size equals its capacity when you add a new element, the vector increases its capacity to accommodate more elements.

```
20 // append 1-10 to integers and display updated size and capacity
21 for (int i : std::views::iota(1, 11)) {
22     integers.push_back(i); // push_back is in vector, deque and list
23     showResult(i, integers.size(), integers.capacity());
24 }
```

```
appended: 1; size: 1; capacity: 1
appended: 2; size: 2; capacity: 2
appended: 3; size: 3; capacity: 3
appended: 4; size: 4; capacity: 4
appended: 5; size: 5; capacity: 6
appended: 6; size: 6; capacity: 6
appended: 7; size: 7; capacity: 9
appended: 8; size: 8; capacity: 9
appended: 9; size: 9; capacity: 9
appended: 10; size: 10; capacity: 13
```

vector Sequence Container (Cont.)

- vector Member Function `push_back`
 - Function `push_back` is available in sequence containers other than `array` and `forward_list`.
 - Sequence containers other than `array` and `vector` also provide a `push_front` function.

vector Sequence Container (Cont.)

- Updated size and capacity After Modifying a vector
 - The C++ standard does NOT specify how a vector grows to accommodate more elements— a time-consuming operation.
 - ◆ Some implementations double the vector's capacity.
 - ◆ Others increase the capacity by 1.5 times, as shown in the output from Visual C++.

vector Sequence Container (Cont.)

- Other vector Sizing Considerations
 - Some programmers allocate a large initial capacity.
 - ◆ If a vector stores a small number of elements, such capacity may waste space.
 - ◆ However, it can greatly improve performance if a program adds many elements to a vector and does not have to reallocate memory to accommodate those elements.
 - This is a classic **space–time trade-off**. **Library implementors must balance the amount of memory used against the time required to perform various vector operations.**
 - It can be wasteful to double a vector's size when more space is needed.
 - ◆ For example, in a vector implementation that doubles the allocated memory when space is needed, a full vector of 1,000,000 elements resizes to accommodate 2,000,000 elements even when only one new element is added.
 - ◆ This leaves 999,999 unused elements.
 - You can use member functions **reserve** to control space usage better.

vector Sequence Container (Cont.)

- Outputting vector Contents with Iterators

- Lines 28–31 output the vector's contents.

- ◆ Line 28 initializes the control variable const-Iterator using vector member function cbegin, which returns a const_iterator to the vector's first element.
 - ◆ We use auto to infer the control variable's type vector<int>::const_iterator which simplifies the code and reduces errors when working with more complex types.

```
26     std::cout << "\nOutput integers using iterators: ";
27
28     for (auto constIterator{integers.cbegin()};
29          constIterator != integers.cend(); ++constIterator) {
30         std::cout << *constIterator << ' ';
31     }
```

Output integers using iterators: 1 2 3 4 5 6 7 8 9 10

vector Sequence Container (Cont.)

- Outputting vector Contents with Iterators

- The loop continues as long as constIterator has not reached the vector's end.
 - ◆ This is determined by comparing constIterator to the result of calling the vector's cend function.
 - ◆ When constIterator equals this value, the loop terminates.
 - ◆ Attempting to dereference an iterator positioned outside its container is a logic error: The iterator returned by end or cend should never be dereferenced or incremented.

```
26      std::cout << "\nOutput integers using iterators: ";  
27  
28      for (auto constIterator{integers.cbegin()};  
29           constIterator != integers.cend(); ++constIterator) {  
30          std::cout << *constIterator << ' ';  
31      }
```

Output integers using iterators: 1 2 3 4 5 6 7 8 9 10

vector Sequence Container (Cont.)

- Outputting vector Contents with Iterators

- Line 30 dereferences constIterator to get the current element's value.
 - ◆ Remember that the iterator acts like a pointer to an element, and operator * is overloaded to return a reference to the element.
 - ◆ The expression ++constIterator (line 29) positions the iterator to the vector's next element.

```
26      std::cout << "\nOutput integers using iterators: ";
27
28      for (auto constIterator{integers.cbegin()};
29           constIterator != integers.cend(); ++constIterator) {
30          std::cout << *constIterator << ' ';
31      }
```

Output integers using iterators: 1 2 3 4 5 6 7 8 9 10

vector Sequence Container (Cont.)

- Outputting vector Contents with Iterators
 - You could replace this loop with the following more straightforward range-based for statement:
**for (auto const& item : integers) {
 std::cout << item << ' ';
}**
 - The range-based for uses iterators “under the hood.”

```
26      std::cout << "\nOutput integers using iterators: ";  
27  
28      for (auto constIterator{integers.cbegin()};  
29          constIterator != integers.cend(); ++constIterator) {  
30          std::cout << *constIterator << ' ';  
31      }
```

Output integers using iterators: 1 2 3 4 5 6 7 8 9 10

vector Sequence Container (Cont.)

- Displaying the vector's Contents in Reverse with `const_reverse_iterators`
 - Lines 36–39 iterate through the vector in reverse.
 - ◆ The vector member functions **`crbegin`** and **`crend`** each return **`const_reverse_iterators`** representing the starting and ending points when iterating through a container in reverse.
 - ◆ Most sequence containers and the ordered associative containers support reverse iteration.
 - ◆ Class `vector` also provides member functions **`rbegin`** and **`rend`** to obtain non-const `reverse_iterators`.

```
33      std::cout << "\nOutput integers in reverse using iterators: ";
34
35      // display vector in reverse order using const_reverse_iterator
36      for (auto reverseIterator{integers.crbegin()};
37           reverseIterator != integers.crend(); ++reverseIterator) {
38          std::cout << *reverseIterator << ' ';
39      }
```

Output integers in reverse using iterators: 10 9 8 7 6 5 4 3 2 1

vector Sequence Container (Cont.)

- shrink_to_fit

- You can call member function **shrink_to_fit** to request that a vector or deque **return unneeded memory to the system**, reducing its capacity to the container's current number of elements.
- According to the C++ standard, **implementations can ignore this request** so that they can perform implementation-specific optimizations.

vector Sequence Container (Cont.)

- vector Element-Manipulation Functions

- The next program illustrates functions for retrieving and manipulating vector elements.
- Line 12 initializes a `vector<int>` with a braced initializer.
 - ◆ In this case, we declared the vector's type as `std::vector`.
 - ◆ The compiler uses class template argument deduction (CTAD) to infer the element type from the initializer list.

```
1 // main.cpp
2 // Testing standard library vector class template
3 // element-manipulation functions.
4 #include <algorithm> // copy algorithm
5 #include <format>
6 #include <iostream>
7 #include <ranges>
8 #include <iterator> // ostream_iterator iterator
9 #include <vector>
10
11 int main() {
12     std::vector values{1, 2, 3, 4, 5}; // class template argument deduction
13     std::vector<int> integers{values.cbegin(), values.cend()};
14     std::ostream_iterator<int> output{std::cout, " "};
```

vector Sequence Container (Cont.)

- vector Element-Manipulation Functions

- Line 13 uses a vector constructor that takes two iterators as arguments to initialize integers with a copy of the elements in the range values.cbegin() up to, but not including, values.cend().

```
1 // main.cpp
2 // Testing standard library vector class template
3 // element-manipulation functions.
4 #include <algorithm> // copy algorithm
5 #include <format>
6 #include <iostream>
7 #include <ranges>
8 #include <iterator> // ostream_iterator iterator
9 #include <vector>
10
11 int main() {
12     std::vector values{1, 2, 3, 4, 5}; // class template argument deduction
13     std::vector<int> integers{values.cbegin(), values.cend()};
14     std::ostream_iterator<int> output{std::cout, " "};
```

vector Sequence Container (Cont.)

- ostream_iterator

- Line 14 defines an ostream_iterator called output that can be used to output integers separated by single spaces via cout.
 - ◆ An ostream_iterator<int> outputs only values of type int.
 - ◆ The constructor's **first argument specifies the output stream**, and the **second argument is a string specifying the separator** for the values output—in this case, a space character.
 - ◆ We use the ostream_iterator (header <iterator>) to output the vector contents in this example.

```
11 int main() {  
12     std::vector values{1, 2, 3, 4, 5}; // class template argument deduction  
13     std::vector<int> integers{values.cbegin(), values.cend()};  
14     std::ostream_iterator<int> output{std::cout, " "};
```

vector Sequence Container (Cont.)

- copy Algorithm

- Line 17 uses standard library algorithm copy (header <algorithm>) to output the contents of integers to the standard output.
- This version of the algorithm takes three arguments.
 - ◆ The first two are iterators that specify the elements to copy from vector integers—from `integers.cbegin()` up to, but not including, `integers.cend()`.
 - These must satisfy input iterator requirements, such as `const_iterator`, enabling values to be read from a container.
 - They must also refer to the same container such that applying `++` to the first iterator repeatedly causes it to reach the second iterator argument eventually.
 - ◆ The third argument specifies where to copy the elements.
 - This must be an output iterator through which a value can be stored or output.
 - The output iterator is an `ostream_iterator` attached to `cout`, so line 17 copies the elements to the standard output.

```
16 | std::cout << "integers contains: ";  
17 | std::copy(integers.cbegin(), integers.cend(), output);
```

integers contains: 1 2 3 4 5

vector Sequence Container (Cont.)

- Common Ranges

- Before C++20, a “range” of container elements was described by iterators specifying the starting position and the one-past-the-end position, typically via calls to a container’s **begin** and **end** (or similar) member functions.
- As of C++20, the C++ standard refers to such ranges as **common ranges**, so they’re not confused with C++20 ranges.
- From this point forward, we’ll use the term “common range” when discussing ranges determined by two iterators, and we’ll use the term “range” when discussing C++20 ranges.

vector Sequence Container (Cont.)

- vector Member Functions front and back
 - Lines 19–20 get the vector's first and last elements via member functions front and back, which are available in most sequence containers.
 - Notice the difference between functions front and begin.
 - ◆ Function `front` returns a reference to the vector's first element, while function `begin` returns an iterator pointing to the vector's first element.
 - ◆ Similarly, function `back` returns a reference to the vector's last element, whereas `end` returns an iterator pointing to the location after the last element.
 - ◆ The results of `front` and `back` are undefined when called on an empty vector.

19
20



```
std::cout << std::format("\nfront: {} \nback: {} \n\n",  
    integers.front(), integers.back());
```

front: 1
back: 5

vector Sequence Container (Cont.)

- Accessing vector Elements

- Lines 22–23 illustrate two ways to access vector (or deque) elements.
 - ◆ Line 22 uses the `[]` operator, which returns a reference to the value at the specified location or a reference to that const value, depending on whether the container is const.
 - ◆ Function `at` (line 23) performs the same operation but **with bounds checking**.
 - The `at` member function first checks its argument and determines whether it's in the vector's bounds.
 - If not, the `at` function throws an `out_of_range` exception.

```
22 integers[0] = 7; // set first element to 7
23 integers.at(2) = 10; // set element at position 2 to
```

vector Sequence Container (Cont.)

- vector Member Function insert

- Line 26 uses one of the several overloaded insert member functions provided by each sequence container (except array, which has a fixed size, and forward_list, which has the function insert_after instead).
 - ◆ Line 26 inserts the value 22 before the element at the location specified by the iterator in the first argument.
 - ◆ Here, the iterator points to the second element, so 22 becomes the second element, and the original second element is now the third.

```
25 // insert 22 as second element
26 integers.insert(integers.cbegin() + 1, 22);
27
28 std::cout << "Contents of vector integers after changes: "
29 std::ranges::copy(integers, output);
```

Contents of vector integers after changes: 7 22 2 10 4 5

vector Sequence Container (Cont.)

- vector Member Function insert
 - Other versions of insert allow
 - ◆ inserting multiple copies of the same value starting at a given position or
 - ◆ inserting a range of values from another container, starting at a given position.
 - The version of insert in line 26 returns an iterator pointing to the inserted item.

vector Sequence Container (Cont.)

- C++20 Ranges Algorithm copy

- Line 29 uses the C++20 copy algorithm from the `std::ranges` namespace to copy the elements of integers to the standard output.
- This version of copy receives only the range to copy and the output iterator representing where to copy the range's elements.
 - ◆ The first argument is an object representing a range of elements with input iterators representing its beginning and end—in this case, a vector.

```
25 // insert 22 as second element
26 integers.insert(integers.cbegin() + 1, 22);
27
28 std::cout << "Contents of vector integers after changes: "
29 std::ranges::copy(integers, output);
```

Contents of vector integers after changes: 7 22 2 10 4 5

vector Sequence Container (Cont.)

- C++20 Ranges Algorithm copy
 - Most pre-C++20 algorithms in the `<algorithm>` header now have C++20 ranges versions in the `std::ranges` namespace.
 - ◆ The `std::ranges` algorithms typically are overloaded with a version that takes a range object and a version that takes an iterator and a sentinel.
 - In C++20 ranges, a sentinel is an object that represents when the end of the container has been reached.
 - Chapter 14 demonstrates many C++20 ranges algorithms.

vector Sequence Container (Cont.)

- vector Member Function erase
 - Lines 31 and 36 use two erase member functions that are available in most sequence and associative containers—except
 - ◆ array, which has a fixed size, and
 - ◆ forward_list, which has the function erase_after instead.

vector Sequence Container (Cont.)

- vector Member Function erase

- Line 31 erases the element at the location specified by its iterator argument—in this case, the first element.
- Line 36 erases elements in the common range specified by the two iterator arguments—in this case, all the elements.
- Line 38 uses the member function empty (defined for all containers and adaptors) to confirm that the vector is empty.

```
31 integers.erase(integers.cbegin()); // erase first element
32 std::cout << "\n\nintegers after erasing first element: ";
33 std::ranges::copy(integers, output);
34
35 // erase remaining elements
36 integers.erase(integers.cbegin(), integers.cend());
37 std::cout << std::format("\nErased all elements: integers {} empty\n",
38 integers.empty() ? "is" : "is not");
```

```
integers after erasing first element: 22 2 10 4 5
Erased all elements: integers is empty
```

vector Sequence Container (Cont.)

- vector Member Function erase
 - Usually, erase destroys the objects it erases.
 - **However, erasing an element that is a pointer to a dynamically allocated object does not delete the object it points to, potentially causing a memory leak.**
 - Again, this is why you should not manage dynamically allocated memory with raw pointers.
 - A `unique_ptr` element would release the dynamically allocated memory.
 - If the element is a `shared_ptr`, the reference count to the dynamically allocated object would be decremented, and the memory would be deleted only if the reference count reached 0.

vector Sequence Container (Cont.)

- vector Member Function insert with Three Arguments (Range insert)
 - Line 41 demonstrates the version of function **insert** that uses **its second and third arguments to specify a common range of elements** to insert into the vector.
 - ◆ Remember that the ending location specifies the position in the sequence after the last element to insert; copying occurs up to, but **NOT including**, this location.
 - ◆ This version of insert returns an iterator pointing to the first item that was inserted.
 - ◆ If nothing was inserted, the function returns its first argument.

```
40 // insert elements from the vector values
41 integers.insert(integers.cbegin(), values.cbegin(), values.cend());
42 std::cout << "\nContents of vector integers before clear: ";
43 std::ranges::copy(integers, output);
```

Contents of vector integers before clear: 1 2 3 4 5

vector Sequence Container (Cont.)

- vector Member Function clear

- Finally, line 46 uses the member function clear to empty the vector.
- **This does not reduce the vector's capacity.**
- Other than array, which is fixed in size, all sequence containers and associative containers provide clear.

```
45 // empty integers; clear empties a collection
46 integers.clear();
47 std::cout << std::format("\nAfter clear, integers {} empty\n",
48     integers.empty() ? "is" : "is not");
49 }
```

After clear, integers is empty

list Sequence Container

- The list sequence container (from header <list>) allows insertion and deletion operations at any location in the container.
- The list container is implemented as a **doubly linked list**—every node in the list contains a pointer to the previous node in the list and to the next node in the list.
 - This enables lists to support **bidirectional iterators** that allow the container to be traversed both forward and backward.
 - Any algorithm that requires **input, output, forward** or **bidirectional iterators** can operate on a list.
- Many list member functions manipulate the elements of the container as an ordered set of elements.
- If most of the insertions and deletions occur at the ends of the container, consider using deque or vector.
 - Recall that **deque's implementation of inserting at the front is more efficient than vector's**.

list Sequence Container (Cont.)

- forward_list Container

- The forward_list sequence container (header <forward_list>) is implemented as a **singly linked list**—every node contains a pointer to the next node in the forward_list.
- This enables a forward_list to **support forward iterators** that allow the container to be traversed in the forward direction.
- Any algorithm that requires **input**, **output** or **forward iterators** can operate on a forward_list.

list Sequence Container (Cont.)

- list Member Functions

- In addition to the common sequence container member functions we have discussed, list provides member functions **splice**, **push_front**, **pop_front**, **emplace_front**, **emplace_back**, **remove**, **remove_if**, **unique**, **merge**, **reverse** and **sort**.
 - ◆ Several of these are list-optimized implementations of standard library algorithms we show in Chapter 14.
 - ◆ Both **push_front** and **pop_front** are also supported by **forward_list** and **deque**.

list Sequence Container (Cont.)

- list Member Functions

- The next program demonstrates several list features.
- Many of the functions presented in the examples of `vector` also can be used with lists.
- We focus on the new features in this example's discussion.

list Sequence Container (Cont.)

- list Member Functions

- Function template printList (lines 11–20) checks whether its list argument is empty (line 13) and, if so, displays an appropriate message.
- Otherwise, printList uses an ostream_iterator and the std::ranges::copy algorithm to copy the list's elements to the standard output.

```
1 // main.cpp
2 // Standard library list class template.
3 #include <algorithm> // copy algorithm
4 #include <iostream>
5 #include <iterator> // ostream_iterator
6 #include <list> // list class-template definition
7 #include <vector>
8
9 // printList function template definition; uses
10 // ostream_iterator and copy algorithm to output list elements
11 template <typename T>
12 void printList(const std::list<T>& items) {
13     if (items.empty()) { // list is empty
14         std::cout << "List is empty";
15     }
16     else {
17         std::ostream_iterator<T> output{std::cout, " "};
18         std::ranges::copy(items, output);
19     }
20 }
```

list Sequence Container (Cont.)

- Creating a list Object

- Line 23 creates a list object capable of storing ints.
- Lines 26–27 use its member function `push_front` to insert integers at the beginning of values.
 - ◆ This member function is specific to classes **forward_list**, **list** and **deque**.

```
22 int main() {  
23     std::list<int> values{}; // create list of ints  
24  
25     // insert items in values  
26     values.push_front(1);  
27     values.push_front(2);  
28     values.push_back(4);  
29     values.push_back(3);  
30  
31     std::cout << "values contains: ";  
32     printList(values);
```

values contains: 2 1 4 3

list Sequence Container (Cont.)

- Creating a list Object

- Lines 28–29 use `push_back` to append integers to `values`.
 - ◆ Function `push_back` is common to all sequence containers **except `array` and `forward_list`**.
- Lines 31–32 show the current contents of `values`.

```
22 int main() {  
23     std::list<int> values{}; // create list of ints  
24  
25     // insert items in values  
26     values.push_front(1);  
27     values.push_front(2);  
28     values.push_back(4);  
29     values.push_back(3);  
30  
31     std::cout << "values contains: ";  
32     printList(values);
```

values contains: 2 1 4 3

list Sequence Container (Cont.)

- list Member Function sort

- Line 34 uses list member function sort to arrange the elements in ascending order.
- A second version of sort allows you to supply a **binary predicate function** that takes two arguments (values in the list), performs a comparison and returns a bool value **indicating whether the first argument should come before the second** in the sorted contents.
 - ◆ This function determines the order in which the elements are sorted.
 - ◆ This version is useful for sorting a list in descending order or customizing how elements are compared to determine the sort order.

```
34 values.sort(); // sort values
35 std::cout << "\nvalues after sorting contains: ";
36 printList(values);
37
```

values after sorting contains: 1 2 3 4

list Sequence Container (Cont.)

- list Member Function splice
 - Line 46 uses list member function splice to **remove the elements in otherValues and insert them into values before the iterator position specified as the first argument.**

```
38 // insert elements of ints into otherValues
39 std::vector<int> ints{2, 6, 4, 8};
40 std::list<int> otherValues{}; // create list of ints
41 otherValues.insert(otherValues.cbegin(), ints.cbegin(), ints.cend());
42 std::cout << "\nAfter insert, otherValues contains: ";
43 printList(otherValues);
44
45 // remove otherValues elements and insert at end of values
46 values.splice(values.cend(), otherValues);
47 std::cout << "\nAfter splice, values contains: ";
48 printList(values);
```

After insert, otherValues contains: 2 6 4 8
After splice, values contains: 1 2 3 4 2 6 4 8

list Sequence Container (Cont.)

- list Member Function splice
 - Function splice with three arguments allows **one element to be removed from the container specified as the second argument from the location specified by the iterator in the third argument.**
 - Function splice with four arguments **uses the last two arguments to specify a common range** to remove from the list in the second argument and insert at the location specified in the first argument.
 - A forward_list provides a similar member function named splice_after.

list Sequence Container (Cont.)

- list Member Function merge

- After inserting more elements in otherValues and sorting both values and otherValues, line 61 uses list member function merge to remove all elements of otherValues and insert them in sorted order into values.
- Both lists must be sorted in the same order before this operation is performed.

```
50 values.sort(); // sort values
51 std::cout << "\nAfter sort, values contains: ";
52 printList(values);
53
54 // insert elements of ints into otherValues
55 otherValues.insert(otherValues.cbegin(), ints.cbegin(), ints.cend());
56 otherValues.sort(); // sort the list
57 std::cout << "\nAfter insert and sort, otherValues contains: ";
58 printList(otherValues);
59
60 // remove otherValues elements and insert into values in sorted order
61 values.merge(otherValues);
62 std::cout << "\nAfter merge:\n  values contains: ";
63 printList(values);
64 std::cout << "\n  otherValues contains: ";
65 printList(otherValues);
```

After sort, values contains: 1 2 2 3 4 4 6 8
After insert and sort, otherValues contains: 2 4 6 8
After merge:
values contains: 1 2 2 2 3 4 4 4 6 6 8 8
otherValues contains: List is empty

list Sequence Container (Cont.)

- list Member Function merge

- After inserting more elements in otherValues and sorting both values and otherValues, line 61 uses list member function merge to remove all elements of otherValues and insert them in sorted order into values.
- Both lists must be sorted in the same order before this operation is performed.
- A second version of merge enables you to supply a **binary predicate function** that takes two arguments (values in the list) and returns a bool value.
 - ◆ The predicate function **specifies the sorting order** used by merge, returning true if the predicate's first argument should be placed before the second.

list Sequence Container (Cont.)

- list Member Function pop_front and pop_back
 - Line 67 uses pop_front to remove the list's first element, and
 - line 68 uses pop_back to remove the last element.
 - ◆ This function is available for sequence containers, except array and forward_list.

```
67 values.pop_front(); // remove element from front
68 values.pop_back(); // remove element from back
69 std::cout << "\nAfter pop_front and pop_back:\n" values contains: ";
70 printList(values);
```

After pop_front and pop_back:
values contains: 2 2 2 3 4 4 4 6 6 8

list Sequence Container (Cont.)

- list Member Function unique
 - Line 72 uses list function unique to **remove duplicate adjacent elements**.
 - ◆ If the list is sorted, all duplicates are eliminated.
 - A second version of unique enables you to supply a predicate function that takes two arguments (values in the list) and returns a bool value specifying whether two elements are equal.

```
72 values.unique(); // remove duplicate elements
73 std::cout << "\nAfter unique, values contains: ";
74 printList(values);
```

After unique, values contains: 2 3 4 6 8

list Sequence Container (Cont.)

- list Member Function swap
 - Line 76 uses list member function swap to exchange the contents of values with the contents of otherValues.
 - This function is available to **all sequence containers** and **associative containers**.

```
76 values.swap(otherValues); // swap elements of values and otherValues
77 std::cout << "\nAfter swap:\n  values contains: ";
78 printList(values);
79 std::cout << "\n  otherValues contains: ";
80 printList(otherValues);
```

After swap:

values contains: List is empty
otherValues contains: 2 3 4 6 8

list Sequence Container (Cont.)

- list Member Functions assign and remove
 - Line 83 uses list member function **assign** (available in all sequence containers) to replace values' contents **with a common range** of elements from otherValues.

```
82 // replace contents of values with elements of otherValues
83 values.assign(otherValues.cbegin(), otherValues.cend());
84 std::cout << "\nAfter assign, values contains: ";
85 printList(values);
86
87 // remove otherValues elements and insert into values in sorted order
88 values.merge(otherValues);
89 std::cout << "\nAfter merge, values contains: ";
90 printList(values);
91
92 values.remove(4); // remove all 4s
93 std::cout << "\nAfter remove(4), values contains: ";
94 printList(values);
95 std::cout << "\n";
96 }
```

After assign, values contains: 2 3 4 6 8

After merge, values contains: 2 2 3 3 4 4 6 6 8 8

After remove(4), values contains: 2 2 3 3 6 6 8 8

list Sequence Container.

- list Member Functions assign and remove
 - A second version of assign replaces the list's contents with the specified number of copies in its first argument of the value specified in its second argument.
 - Line 92 uses list member function remove to delete all copies of the value 4 from the list.

```
82 // replace contents of values with elements of otherValues
83 values.assign(otherValues.cbegin(), otherValues.cend());
84 std::cout << "\nAfter assign, values contains: ";
85 printList(values);
86
87 // remove otherValues elements and insert into values in sorted order
88 values.merge(otherValues);
89 std::cout << "\nAfter merge, values contains: ";
90 printList(values);
91
92 values.remove(4); // remove all 4s
93 std::cout << "\nAfter remove(4), values contains: ";
94 printList(values);
95 std::cout << "\n";
96 }
```

After assign, values contains: 2 3 4 6 8

After merge, values contains: 2 2 3 3 4 4 6 6 8 8

After remove(4), values contains: 2 2 3 3 6 6 8 8

deque Sequence Container

- Class deque (header <deque>)—short for “**double-ended queue**”—provides many benefits of vectors and lists in one container.
 - It's implemented to provide **efficient indexed access** (using subscripting) for reading and modifying elements, like a vector or array.
 - It also provides **efficient insertion and deletion at its front and back**, like a list.
 - Class deque **supports random-access iterators**, so deques can be used with all standard library algorithms.
 - A common use of a deque is to maintain a first-in, first-out queue of elements. In fact, a deque is the **default container for the queue adaptor**.

deque Sequence Container

- Additional storage for a deque can be allocated at either end in blocks of memory that are typically maintained as a built-in array of pointers to those blocks.
 - Due to a deque's noncontiguous memory layout, its iterators must be more “intelligent” than those for vectors, arrays or built-in arrays.
 - A deque provides the same operations as vector, but like list, adds member functions **push_front** and **pop_front** for **efficient insertion** and **deletion** at the beginning of the deque.

deque Sequence Container (Cont.)

- The next program demonstrates several deque features. Many functions presented in the examples of vector and list also can be used with deque.
- Line 9 instantiates a deque that can store double values, then lines 13–15 use member functions `push_front` and `push_back` to insert elements at its beginning and end.

```
1 // main.cpp
2 // Standard library deque class template.
3 #include <algorithm> // copy algorithm
4 #include <deque> // deque class-template definition
5 #include <iostream>
6 #include <iterator> // ostream iterator
7
8 int main() {
9     std::deque<double> values; // create deque of doubles
10    std::ostream_iterator<double> output{std::cout, " "};
11
12    // insert elements in values
13    values.push_front(2.2);
14    values.push_front(3.5);
15    values.push_back(1.1);
```

deque Sequence Container (Cont.)

- Lines 20–22 use the **subscript operator** to retrieve **each element's value** for output.
 - The loop condition uses member function `size` to ensure that we do not attempt to access an element outside the deque's bounds.
 - We use a counter-controlled for loop here only to demonstrate the `[]` operator.
 - **Generally, you should use the range-based for to process all the elements of a container.**

```
17      std::cout << "values contains: ";  
18  
19      // use subscript operator to obtain elements of values  
20      for (size_t i{0}; i < values.size(); ++i) {  
21          std::cout << values[i] << ' ';  
22      }
```

deque Sequence Container.

- Line 24 uses `pop_front` to **remove the deque's first element**.
- Line 29 uses the **subscript operator to obtain an lvalue**, which we use to assign a new value to element 1 of the deque.

```
24 values.pop_front(); // remove first element
25 std::cout << "\nAfter pop_front, values contains: ";
26 std::ranges::copy(values, output);
27
28 // use subscript operator to modify element at location 1
29 values[1] = 5.4;
30 std::cout << "\nAfter values[1] = 5.4, values contains: ";
31 std::ranges::copy(values, output);
32 std::cout << "\n";
33 }
```

Associative Containers

- The associative containers provide direct access to store and retrieve elements via keys.
- The four ordered associative containers are multiset, set, multimap and map.
 - Each of these maintains its keys in sorted order.
- There are also four corresponding unordered associative containers—`unordered_multiset`, `unordered_set`, `unordered_multimap` and `unordered_map`
 - that offer most of the same capabilities as their ordered counterparts.
- The primary difference between ordered and unordered associative containers is that **unordered ones do not maintain their keys in sorted order.**
- **If your keys are NOT comparable, you must use the unordered containers.**
- Here we focus on the ordered associative containers.

Associative Containers (Cont.)

- For cases in which it's not necessary to maintain keys in sorted order, **the unordered associative containers offer better search performance via hashing**— $O(1)$ with a worstcase of $O(n)$ vs. $O(\log n)$ for the ordered associative containers.
 - However, this requires the keys to be **hashable**.
 - For hashable-type requirements, see the documentation for `std::hash`.

Associative Containers.

- Iterating through an **ordered associative container** traverses it in the sort order for that container.
- Classes **multiset** and **set** provide operations for manipulating sets of values where the values themselves are the keys.
 - The primary difference between a multiset and a set is that a multiset allows duplicate keys and a set does not.
- Classes **multimap** and **map** provide operations for manipulating values associated with keys (these values are sometimes referred to as **mapped values**).
 - The primary difference between a multimap and a map is that a multimap allows multiple values for the same key and a map allows only one value per key.

multiset Associative Container

- The multiset ordered associative container (from header `<set>`) provides **fast storage and retrieval of keys and allows duplicate keys**.
- The elements' ordering is determined by a **comparator function object**.
 - A function object is an instance of a class that **has an overloaded parentheses operator**, allowing the object to be “called” like a function.
 - For example, in an integer multiset, elements can be sorted in ascending order by ordering the keys with comparator function object **`less<int>`**, which knows how to compare two int values to determine whether the first is less than the second.
 - This enables an integer multiset to order its elements in ascending order.
 - Chapter 14 discusses function objects in detail. Here, we'll simply show how to use `less<int>` when declaring ordered associative containers.

multiset Associative Container (Cont.)

- The data type of the keys in all ordered associative containers must support comparison based on the **comparator function object**—keys sorted with **less<T>** must support comparison with **operator<**.
- If the keys used in the ordered associative containers are of user-defined data types, those types must supply comparison operators.
- A multiset supports **bidirectional iterators**.
- If the order of the keys is not important, consider **unordered_multiset** (header `<unordered_set>`), but keep in mind that it supports only forward iterators.

multiset Associative Container (Cont.)

- Creating a multiset
 - The next program demonstrates the multiset ordered associative container with int keys that are sorted in ascending order.
 - Containers multiset and set provide the same basic functionality.

multiset Associative Container (Cont.)

- Creating a multiset

- Line 12 creates the multiset, using the function object `std::less<int>` to specify the keys' sort order.
- The compiler knows this multiset contains `int` values, so you can specify `std::less<int>` simply as `std::less<>`—the compiler will infer that `std::less` compares `int` values.

```
1 // main.cpp
2 // Standard library multiset class template
3 #include <algorithm> // copy algorithm
4 #include <format>
5 #include <iostream>
6 #include <iterator> // ostream_iterator
7 #include <ranges>
8 #include <set> // multiset class-template definition
9 #include <vector>
10
11 int main() {
12     std::multiset<int, std::less<int>> ints{}; // multiset of int values
13     std::cout << std::format("15s in ints: {}\n", ints.count(15));
```

15s in ints: 0

multiset Associative Container (Cont.)

- Creating a multiset

- Also, `std::less<>` is the default for a multiset, so line 12 can be simplified as
`std::multiset<int> ints{}; // multiset of int values`

```
1 // main.cpp
2 // Standard library multiset class template
3 #include <algorithm> // copy algorithm
4 #include <format>
5 #include <iostream>
6 #include <iterator> // ostream_iterator
7 #include <ranges>
8 #include <set> // multiset class-template definition
9 #include <vector>
10
11 int main() {
12     std::multiset<int, std::less<int>> ints{}; // multiset of int values
13     std::cout << std::format("15s in ints: {}\n", ints.count(15));
```

15s in ints: 0

multiset Associative Container (Cont.)

- multiset Member Function **count**
 - Line 13 uses function **count** (available to all associative containers) to count **the number of occurrences of the value 15** currently in the multiset.

```
1 // main.cpp
2 // Standard library multiset class template
3 #include <algorithm> // copy algorithm
4 #include <format>
5 #include <iostream>
6 #include <iterator> // ostream_iterator
7 #include <ranges>
8 #include <set> // multiset class-template definition
9 #include <vector>
10
11 int main() {
12     std::multiset<int, std::less<int>> ints{}; // multiset of int values
13     std::cout << std::format("15s in ints: {}\n", ints.count(15));
```

15s in ints: 0

multiset Associative Container (Cont.)

- multiset Member Function insert
 - Lines 16–17 use one of the several overloaded versions of the insert member function to add the value 15 to the multiset twice.

```
15      std::cout << "\nInserting two 15s into ints\n";  
16      ints.insert(15); // insert 15 in ints  
17      ints.insert(15); // insert 15 in ints  
18      std::cout << std::format("15s in ints: {}\n\n", ints.count(15));
```

```
Inserting two 15s into ints  
15s in ints: 2
```

multiset Associative Container (Cont.)

- multiset Member Function insert
 - A second version of insert takes an iterator and a value as arguments and begins the search for the insertion point from the iterator position specified.
 - A third version of insert takes two iterators that specify a common range to add to the multiset from another container.
 - For several other overloads, see <https://en.cppreference.com/w/cpp/container/multiset/insert>

multiset Associative Container (Cont.)

- multiset Member Function **find**
 - Lines 21–28 use member function **find** (line 22), which is **available to all associative containers**, to search for the first locations of the values 15 and 20 in the multiset.
 - The range-based for loop iterates through each item in {15, 20}, which creates an `initializer_list`.

```
20 // search for 15 and 20 in ints; find returns an iterator
21 for (int i : {15, 20}) {
22     if (auto result{ints.find(i)}; result != ints.end()) {
23         std::cout << std::format("Found {} in ints\n", i);
24     }
25     else {
26         std::cout << std::format("Did not find {} in ints\n", i);
27     }
28 }
```

```
Found 15 in ints
Did not find 20 in ints
```

multiset Associative Container (Cont.)

- multiset Member Function `find`
 - Function `find` returns either an iterator or a `const_iterator`, depending on whether the multiset is `const`.
 - ◆ The iterator points to the location at which the value is found.
 - ◆ If the value is not found, `find` returns an iterator or a `const_iterator` equal to the value returned by the container's `end` member function.
 - Usually, you'd use `find` if you need to use the iterator that points to the found element.
 - ◆ Here, we could have used the `count` member function—if it returns 0, the item is not in the multiset.

multiset Associative Container (Cont.)

- multiset Member Function contains (C++20)
 - Lines 31–38 use the C++20 member function contains (line 32) to determine whether the values 15 and 20 are in the multiset.
 - This function, which is **available to all associative containers**, returns a bool indicating whether the value is present in the container.

```
30 // search for 15 and 20 in ints; contains returns a bool
31 for (int i : {15, 20}) {
32     if (ints.contains(i)) {
33         std::cout << std::format("Found {} in ints\n", i);
34     }
35     else {
36         std::cout << std::format("Did not find {} in ints\n", i);
37     }
38 }
```

Found 15 in ints
Did not find 20 in ints

multiset Associative Container (Cont.)

- Inserting Elements of Another Container into a multiset
 - Line 42 uses member function insert to insert a vector's elements into the multiset, then line 44 copies the multiset's elements to the standard output.
 - ◆ The values display in ascending order because the multiset is an ordered container that maintains its elements in ascending order by default.
 - In line 44, the expression `std::ostream_iterator<int>{std::cout, " "}` creates a temporary ostream_iterator and immediately passes it to copy.
 - ◆ We chose this approach because the ostream_iterator is used only once in this example.

```
40 // insert elements of vector values into ints
41 const std::vector values{7, 22, 9, 1, 18, 30, 100, 22, 85, 13};
42 ints.insert(values.cbegin(), values.cend());
43 std::cout << "\nAfter insert, ints contains:\n";
44 std::ranges::copy(ints, std::ostream_iterator<int>{std::cout, " "});
```

After insert, ints contains:
1 7 9 13 15 15 18 22 22 30 85 100

multiset Associative Container (Cont.)

- multiset Member Functions `lower_bound` and `upper_bound`
 - Line 49 uses functions `lower_bound` and `upper_bound`, **available in all ordered associative containers**, to locate the **earliest occurrence** of the value 22 in the multiset and the element after the last occurrence of the value 22 in the multiset.
 - Both functions **return iterators or const_iterators** pointing to the appropriate location, or they return the **end iterator** if the value is not in the multiset.
 - Together, the lower bound and upper bound represent the common range of elements containing the value 22.

```
46 // determine lower and upper bound of 22 in ints
47 std::cout << std::format(
48     "\n\nlower_bound(22): {} \nupper_bound(22): {} \n\n",
49     *ints.lower_bound(22), *ints.upper_bound(22));
```

```
lower_bound(22): 22
upper_bound(22): 30
```

multiset Associative Container (Cont.)

- pair Objects and multiset Member Function `equal_range`

- The multiset function **`equal_range`** returns a **pair** containing the results of calling both **`lower_bound`** and **`upper_bound`**.
 - ◆ A pair associates two values.
- Line 52 creates and initializes a pair object called `p`.
 - ◆ We use `auto` to infer the variable's type from its initializer.
 - ◆ The pair returned by `equal_range`, which will contain two iterators or `const_iterators`, depending on whether the multiset is `const`.
 - ◆ A pair contains two public data members called `first` and `second`—their types depend on the pair's initializers.

```
51 // use equal_range to determine lower and upper bound of 22 in ints
52 auto p{ints.equal_range(22)};
53 std::cout << std::format(
54     "lower_bound(22): {} \nupper_bound(22): {} \n",
55     *(p.first), *(p.second));
56 }
```

```
lower_bound(22): 22
upper_bound(22): 30
```

multiset Associative Container (Cont.)

- pair Objects and multiset Member Function `equal_range`

- Line 52 uses `equal_range` to determine the `lower_bound` and `upper_bound` of 22 in the multiset.
- Line 55 uses `p.first` and `p.second` to access the `lower_bound` and `upper_bound`.
 - ◆ We dereferenced the iterators to output the values at the locations returned from `equal_range`.
 - ◆ Though we did not do so here, you should always ensure that the iterators returned by `lower_bound`, `upper_bound` and `equal_range` **are not equal to the container's end iterator before dereferencing them.**

```
51 // use equal_range to determine lower and upper bound of 22 in ints
52 auto p{ints.equal_range(22)};
53 std::cout << std::format(
54     "lower_bound(22): {} \nupper_bound(22): {} \n",
55     *(p.first), *(p.second));
56 }
```

```
lower_bound(22): 22
upper_bound(22): 30
```

multiset Associative Container.

- Heterogeneous Lookup

- Before C++14, when searching for a key in an associative container, the argument provided to a search function like `find` was required to have the container's key type.
 - ◆ For example, if the key type were `string`, you could pass `find` a pointer-based `string` to locate in the container.
 - ◆ In this case, the argument would be converted into a temporary object of the key type (`string`), then passed to `find`.
- In C++14 and higher, the argument to `find` (and other similar functions) can be of any type, provided that **there are overloaded comparison operators that can compare values of the argument's type to values of the container's key type**.
 - ◆ If there are, **no temporary objects will be created**. This is known as **heterogeneous lookup**.

set Associative Container

- The set associative container (from header `<set>`) is used for **fast storage and retrieval of unique keys**.
- The implementation of a set is identical to that of a multiset, except that a set must have unique keys.
 - When a duplicate is inserted, it is ignored. This is the intended mathematical behavior of a set, so it's not considered an error.
- A set supports **bidirectional iterators**.
- If the order of the keys is not important, consider `unordered_set` (header `<unordered_set>`), but keep in mind that it **supports only forward iterators**, and its keys must be **hashable**.

set Associative Container (Cont.)

- Creating a set

- The next program demonstrates a set of doubles.
- Line 10 creates the set, using **class template argument deduction (CTAD)** to infer the element type, which is equivalent to
`std::set<double, std::less<double>> doubles{2.1, 4.2, 9.5, 2.1, 3.7};`

set Associative Container (Cont.)

- Creating a set

- The set's `initializer_list` constructor inserts all the elements into the set.
- Line 14 uses the `std::ranges` algorithm **copy** to output the set's contents.
 - ◆ Notice that the value 2.1, which appeared twice in the initializer list, appears only once in doubles.
 - ◆ Again, a set does not allow duplicates.

```
1 // main.cpp
2 // Standard library set class template.
3 #include <algorithm>
4 #include <format>
5 #include <iostream>
6 #include <iterator> // ostream_iterator
7 #include <set>
8
9 int main() {
10     std::set doubles{2.1, 4.2, 9.5, 2.1, 3.7}; // CTAD
11
12     std::ostream_iterator<double> output{std::cout, " "};
13     std::cout << "doubles contains: ";
14     std::ranges::copy(doubles, output);
```

doubles contains: 2.1 3.7 4.2 9.5

set Associative Container (Cont.)

- Inserting a New Value into a set
 - Line 19 defines and initializes a pair to store the result of calling set member function **insert**.
 - ◆ The pair contains **an iterator pointing to the item** in the set and **a bool indicating whether the item was inserted**—true if the item was not previously in the set and false otherwise.
 - ◆ In this case, line 19 uses function insert to place the value 13.8 in the set and returns a pair in which **p.first points to the value 13.8 in the set and p.second is true** because the value was inserted.

```
16 // insert 13.8 in doubles; insert returns pair in which
17 // p.first represents location of 13.8 in doubles and
18 // p.second represents whether 13.8 was inserted
19 auto p{doubles.insert(13.8)}; // value not in set
20 std::cout << std::format("\n{ } inserted\n", *(p.first),
21     (p.second ? "was" : "was not"));
22 std::cout << "doubles contains: ";
23 std::ranges::copy(doubles, output);
```

13.8 was inserted
doubles contains: 2.1 3.7 4.2 9.5 13.8

set Associative Container.

- Inserting an Existing Value into a set
 - Line 26 attempts to insert 9.5, which is already in the set.
 - ◆ The output shows that 9.5 was not inserted because sets don't allow duplicate keys.
 - ◆ In this case, p.first in the returned pair points to the existing 9.5 in the set and p.second is false.

```
25 // insert 9.5 in doubles
26 p = doubles.insert(9.5); // value already in set
27 std::cout << std::format("\n{} {} inserted\n", *(p.first),
28   (p.second ? "was" : "was not"));
29 std::cout << "doubles contains: ";
30 std::ranges::copy(doubles, output);
31 std::cout << "\n";
32 }
```

9.5 was not inserted
doubles contains: 2.1 3.7 4.2 9.5 13.8

multimap Associative Container

- The multimap associative container is used for **fast storage and retrieval of keys and associated values** (often called key–value pairs).
 - Many of the functions used with multisets and sets are also used with multimaps and maps.
- The elements of multimaps and maps are pairs of keys and values.
 - When inserting into a multimap or map, you use a pair object containing the key and the value.
- The ordering of the keys is determined by a **comparator function object**.
 - For example, in a multimap that uses integers as the key type, keys can be sorted in ascending order by ordering them with comparator function object **less<int>**.

multimap Associative Container (Cont.)

- In a multimap, multiple values can be associated with a single key: This is called a **one-to-many relationship**.
 - For example, in a credit-card transaction-processing system, one credit-card account can have many associated transactions;
 - in a university, one student can take many courses, and one professor can teach many students;
 - in the military, one rank (like “private”) has many people.
- A multimap supports **bidirectional iterators**.

multimap Associative Container (Cont.)

- Creating a multimap Containing Key–Value Pairs
 - The next program demonstrates the multimap associative container (header `<map>`).
 - A multimap is implemented to efficiently locate all values paired with a given key.
 - If the order of the keys is not important, you can use `unordered_multimap` (header `<unordered_map>`) instead.

multimap Associative Container (Cont.)

- Creating a multimap Containing Key–Value Pairs
 - Line 8 creates a multimap in which the key type is int, the type of a key's associated value is double, and the elements are ordered in ascending order by default. This is equivalent to `std::multimap<int, double, std::less<int>> pairs{};`

```
1 // main.cpp
2 // Standard library multimap class template.
3 #include <format>
4 #include <iostream>
5 #include <map> // multimap class-template definition
6
7 int main() {
8     std::multimap<int, double> pairs{}; // create multimap
9     std::cout << std::format("Number of 15 keys in pairs: {}\n",
10                             pairs.count(15));
```

Number of 15 keys in pairs: 0

multimap Associative Container (Cont.)

- Counting the Number of Key–Value Pairs for a Specific Key
 - Line 10 uses member function **count** to determine the number of key–value pairs with a key of 15—in this case, 0 since the container is currently empty.

```
1 // main.cpp
2 // Standard library multimap class template.
3 #include <format>
4 #include <iostream>
5 #include <map> // multimap class-template definition
6
7 int main() {
8     std::multimap<int, double> pairs{}; // create multimap
9     std::cout << std::format("Number of 15 keys in pairs: {}\n",
10                             pairs.count(15));
```

Number of 15 keys in pairs: 0

multimap Associative Container (Cont.)

- Inserting Key–Value Pairs

- Line 13 uses member function **insert** to add a new key–value pair to the multimap.
- Standard library function **make_pair** creates a pair object, inferring the types of its arguments. In this case,
 - ◆ first represents a key (15) of type int, and
 - ◆ second represents a value (99.3) of type double.
- Line 14 inserts another pair object with the key 15 and the value 2.7.
- Then lines 15–16 output the number of pairs with key 15.

```
12 // insert two pairs
13 pairs.insert(std::make_pair(15, 99.3));
14 pairs.insert(std::make_pair(15, 2.7));
15 std::cout << std::format("Number of 15 keys in pairs: {}\n\n",
16     pairs.count(15));
```

Number of 15 keys in pairs: 2

multimap Associative Container (Cont.)

- Inserting Key–Value Pairs with Braced Initializers Rather Than `make_pair`
 - You can use **braced initialization for pair objects**, so lines 13–14 can be simplified as
`pairs.insert({15, 99.3});`
`pairs.insert({15, 2.7});`
 - Lines 19–23 insert five additional pairs into the multimap.

```
18 // insert five pairs
19 pairs.insert({30, 111.11});
20 pairs.insert({10, 22.22});
21 pairs.insert({25, 33.333});
22 pairs.insert({20, 9.345});
23 pairs.insert({5, 77.54});
```

multimap Associative Container (Cont.)

- Inserting Key–Value Pairs with Braced Initializers Rather Than `make_pair`
 - Lines 28–30 output the keys and values.
 - We infer the type of the loop's control variable—in this case, a pair containing an `int` key and a `double` value.
 - Line 29 accesses each pair's members. Notice that **the keys appear in ascending order because the multimap maintains the keys in ascending order.**

```
25
26
27 // walk through elements of pairs
28 for (const auto& mapItem : pairs) {
29     std::cout << std::format("{}\t{}\n", mapItem.first, mapItem.second);
30 }
31 }
```

Multimap pairs contains:

Key	Value
5	77.54
10	22.22
15	99.3
15	2.7
20	9.345
25	33.333
30	111.11

multimap Associative Container.

- Brace Initializing a Key–Value Pair Container
 - This example used separate calls to member function insert to place key–value pairs in a multimap.
 - If you know the key–value pairs in advance, you can use braced initialization when creating the multimap.
 - For example, the following statement initializes a multimap with three key–value pairs that are represented by the sublists in the main initializer list:

```
std::multimap<int, double> pairs{  
    {10, 22.22}, {20, 9.345}, {5, 77.54} };
```

map Associative Container

- The map associative container (from header `<map>`) performs **fast storage and retrieval of unique keys and associated values**.
 - A single value can be associated with each unique key.
 - This is called a one-to-one mapping.
- For example, a company that uses unique employee numbers, such as 100, 200 and 300, might have a map that associates employee numbers with their telephone extensions—4321, 4115 and 5217, respectively.
- **Providing the key in a map's subscript operator [] locates the value associated with that key in the map.**
- If the order of the keys is not important, consider an `unordered_map` (header `<unordered_map>`), but keep in mind that the keys must be **hashable**.

map Associative Container (Cont.)

- The next program demonstrates a map.

```
1 // main.cpp
2 // Standard library class map class template.
3 #include <iostream>
4 #include <format>
5 #include <map> // map class-template definition
6
7 int main() {
8     // create a map; duplicate keys are ignored
9     std::map<int, double> pairs{{15, 2.7}, {30, 111.11}, {5, 1010.1},
10                                {10, 22.22}, {25, 33.333}, {5, 77.54}, {20, 9.345}, {15, 99.3}};
11
12     // walk through elements of pairs
13     std::cout << "pairs contains:\nKey\tValue\n";
14     for (const auto& pair : pairs) {
15         std::cout << std::format("{}\t{}\n", pair.first, pair.second);
16     }
17
18     pairs[25] = 9999.99; // use subscripting to change value for key 25
19     pairs[40] = 8765.43; // use subscripting to insert value for key 40
20
21     // walk through elements of pairs
22     std::cout << "\nAfter updates, pairs contains:\nKey\tValue\n";
23     for (const auto& pair : pairs) {
24         std::cout << std::format("{}\t{}\n", pair.first, pair.second);
25     }
26 }
```

pairs contains:

Key	Value
5	10
10.1	
10	22.22
15	2.7
20	9.345
25	33.333
30	111.11

After updates, pairs contains:

Key	Value
5	1010.1
10	22.22
15	2.7
20	9.345
25	9999.99
30	111.11
40	8765.43

map Associative Container (Cont.)

- In lines 9–10, only six of the initial eight key–value pairs are inserted because two have duplicate keys.
- Unlike similar data structures in some programming languages, **inserting two key–value pairs with the same key does not replace the first key’s value with that of the second.**
 - If you insert a key–value pair for which the key is already in the map, the key–value pair is ignored.
- Lines 13–16 (and lines 23–25) walk through elements of pairs using range-based for.

```
8 // create a map; duplicate keys are ignored
9 std::map<int, double> pairs{{15, 2.7}, {30, 111.11}, {5, 1010.1},
10 {10, 22.22}, {25, 33.333}, {5, 77.54}, {20, 9.345}, {15, 99.3}};
11
12 // walk through elements of pairs
13 std::cout << "pairs contains:\nKey\tValue\n";
14 for (const auto& pair : pairs) {
15     std::cout << std::format("{}\t{}\n", pair.first, pair.second);
16 }
```

map Associative Container.

- Lines 18–19 use the [] map subscript operator.
 - When the subscript is a key that's in the map, the operator **returns a reference to the associated value**.
 - When the subscript is **a key that's not in the map**, the subscript operator **inserts** a new key–value pair in the map, consisting of the **specified key and the default value** for the container's value type.
 - Line 18 replaces the value for the key 25 (previously 33.333, as specified in line 10) with a new value, 9999.99.
 - Line 19 inserts a new key–value pair in the map.

```
18 pairs[25] = 9999.99; // use subscripting to change value for key 25
19 pairs[40] = 8765.43; // use subscripting to insert value for key 40
20
21 // walk through elements of pairs
22 std::cout << "\nAfter updates, pairs contains:\nKey\tValue\n";
23 for (const auto& pair : pairs) {
24     std::cout << std::format("{}\t{}\n", pair.first, pair.second);
25 }
```

Container Adaptors

- The three container adaptors are `stack`, `queue` and `priority_queue`.
- Container adaptors **do NOT provide a data-structure implementation** in which elements can be stored and **do NOT support iterators**.
- With an adaptor class, you can choose the underlying sequence container or use the adaptor's default choice—`deque` for stacks and queues, or `vector` for `priority_queue`.
- The adaptor classes provide member functions **push** and **pop**:
 - **push properly inserts an element** into an adaptor's underlying container.
 - **pop properly removes an element** from an adaptor's underlying container.

stack Adaptor

- Class `stack` (from header `<stack>`) enables insertions into and deletions from the underlying container at one end (the top).
- So, a stack is commonly referred to as a **last-in, first-out data structure**.
- A stack can be implemented with a vector, list or deque.
 - By default, a stack is implemented with a **deque**.

stack Adaptor (Cont.)

- The stack operations are
 - **push** to insert an element at the stack's top—implemented by calling the underlying container's `push_back` member function,
 - `emplace` to construct an element in place at the stack's top,
 - **pop** to remove the stack's top element—implemented by calling the underlying container's `pop_back` member function,
 - **top** to get a reference to the stack's top element—implemented by calling the underlying container's `back` member function,
 - **empty** to determine whether the stack is empty—implemented by calling the underlying container's `empty` member function, and
 - **size** to get the stack's number of elements—implemented by calling the underlying container's `size` member function.

stack Adaptor (Cont.)

- The next program demonstrates stack.
- This example creates three stacks of ints, using a deque (line 26), a vector (line 27) and a list (line 28) as the underlying data structure.

```
25 int main() {  
26     std::stack<int> dequeStack{}; // uses a deque by default  
27     std::stack<int, std::vector<int>> vectorStack{}; // use a vector  
28     std::stack<int, std::list<int>> listStack{}; // use a list
```

stack Adaptor (Cont.)

- The generic lambda pushElements (lines 10–15) pushes 0–9 onto a stack.
 - Line 12 uses function push (available in each adaptor class) to place an integer on top of the stack.
 - Line 13 uses stack function top to retrieve the stack's top element for output.
 - Function top does not remove the top element.
- Lines 32, 34 and 36 call this lambda for each stack.

```
9 // pushElements generic lambda to push values onto a stack
10 auto pushElements = [](auto& stack) {
11     for (auto i : std::views::iota(0, 10)) {
12         stack.push(i); // push element onto stack
13         std::cout << stack.top() << ' '; // view (and display) top element
14     }
15 };
```

stack Adaptor (Cont.)

- Lines 32, 34 and 36 call this lambda for each stack.

```
25 int main() {  
26     std::stack<int> dequeStack{}; // uses a deque by default  
27     std::stack<int, std::vector<int>> vectorStack{}; // use a vector  
28     std::stack<int, std::list<int>> listStack{}; // use a list  
29  
30     // push the values 0-9 onto each stack  
31     std::cout << "Pushing onto dequeStack: ";  
32     pushElements(dequeStack);  
33     std::cout << "\nPushing onto vectorStack: ";  
34     pushElements(vectorStack);  
35     std::cout << "\nPushing onto listStack: ";  
36     pushElements(listStack);  
}
```

```
Pushing onto dequeStack: 0 1 2 3 4 5 6 7 8 9  
Pushing onto vectorStack: 0 1 2 3 4 5 6 7 8 9  
Pushing onto listStack: 0 1 2 3 4 5 6 7 8 9
```

stack Adaptor.

- The generic lambda popElements (lines 18–23) pops the elements off a stack.
 - Line 20 uses stack function top to retrieve the stack's top element for output.
 - Line 21 uses function pop, available in each adaptor class, to remove the stack's top element.
 - ◆ Function pop does not return a value. So, you must call top to obtain the top element's value before you pop that element from the stack.

```
17 // popElements generic lambda to pop elements off a stack
18 auto popElements = [](auto& stack) {
19     while (!stack.empty()) {
20         std::cout << stack.top() << ' '; // view (and display) top element
21         stack.pop(); // remove top element
22     }
23 };
```

stack Adaptor.

- Lines 40, 42 and 44 call this lambda for each stack.

```
38 // display and remove elements from each stack
39 std::cout << "\n\nPopping from dequeStack: ";
40 popElements(dequeStack);
41 std::cout << "\n\nPopping from vectorStack: ";
42 popElements(vectorStack);
43 std::cout << "\n\nPopping from listStack: ";
44 popElements(listStack);
45 std::cout << "\n";
46 }
```

```
Popping from dequeStack: 9 8 7 6 5 4 3 2 1 0
Popping from vectorStack: 9 8 7 6 5 4 3 2 1 0
Popping from listStack: 9 8 7 6 5 4 3 2 1 0
```

queue Adaptor

- A queue is similar to a waiting line. The first in line is the first removed. So, a queue is referred to as a **first-in, first-out (FIFO) data structure**.
- Class queue (from header <queue>) enables **insertions only at the back** of the underlying data structure and **deletions only from the front**.
- A queue stores its elements in objects of the **list** or **deque** sequence containers—deque is the default.

queue Adaptor (Cont.)

- The common queue operations are
 - **push** to insert an element at the queue's back—implemented by calling the underlying container's `push_back` member function,
 - `emplace` to construct an element in place at the queue's top,
 - **pop** to remove the element at the queue's front—implemented by calling the underlying container's `pop_front` member function,
 - **front** to get a reference to the queue's first element—implemented by calling the underlying container's `front` member function,
 - **back** to get a reference to the queue's last element—implemented by calling the underlying container's `back` member function,
 - **empty** to determine whether the queue is empty—this calls the underlying container's `empty` member function, and
 - **size** to get the queue's number of elements—this calls the underlying container's `size` member function.

queue Adaptor (Cont.)

- The next program demonstrates the queue adaptor class.
 - Line 7 instantiates a queue of doubles.
 - Lines 10–12 use push to add elements to the queue.

```
1 // main.cpp
2 // Standard library queue adaptor class template.
3 #include <iostream>
4 #include <queue> // queue adaptor definition
5
6 int main() {
7     std::queue<double> values{}; // queue with doubles
8
9     // push elements onto queue values
10    values.push(3.2);
11    values.push(9.8);
12    values.push(5.4);
```

queue Adaptor.

- The while statement in lines 17–20 uses `empty` (available in all containers) to determine whether the queue is empty (line 17).
 - While there are more elements in the queue, line 18 uses `front` to read (but not remove) the queue's first element for output.
 - Line 19 removes the queue's first element with `pop`, which is available in all adaptor classes.

```
14  std::cout << "Popping from values: ";
15
16  // pop elements from queue
17  while (!values.empty()) {
18      std::cout << values.front() << ' '; // view front element
19      values.pop(); // remove element
20  }
21
22  std::cout << "\n";
23 }
```

Popping from values: 3.2 9.8 5.4

priority_queue Adaptor

- Class `priority_queue` (from header `<queue>`) enables **insertions in sorted order** into the underlying data structure and **deletions from the front** of the underlying data structure.
 - By default, a `priority_queue` stores its elements in a **vector**.
 - Elements added to a `priority_queue` are inserted in priority order, such that the highest-priority element will be the first element removed.
 - This is usually accomplished by arranging the elements as a **heap** that always maintains its highest-priority element at the front of the data structure.
 - Element comparisons are performed with **comparator function object `less<T>` by default**.

priority_queue Adaptor (Cont.)

- The `priority_queue` operations include
 - **push** to insert an element at the appropriate location based on priority order.
 - **emplace** to construct an element in place and re-sort the `priority_queue` into priority order,
 - **pop** to remove the `priority_queue`'s highest-priority element.
 - **top** to get a reference to the `priority_queue`'s top element—implemented by calling the underlying container's `front` member function.
 - **empty** to determine whether the `priority_queue` is empty—implemented by calling the underlying container's `empty` member function.
 - **size** to get the `priority_queue`'s number of elements—implemented by calling the underlying container's `size` member function.

priority_queue Adaptor (Cont.)

- The next program demonstrates priority_queue.
- Line 7 instantiates a priority_queue of doubles and uses a vector as the underlying data structure.
- Lines 10–12 use member function **push** to add elements to the priority_queue.

```
1 // main.cpp
2 // Standard library priority_queue adaptor class.
3 #include <iostream>
4 #include <queue> // priority_queue adaptor definition
5
6 int main() {
7     std::priority_queue<double> priorities; // create priority_queue
8
9     // push elements onto priorities
10    priorities.push(3.2);
11    priorities.push(9.8);
12    priorities.push(5.4);
```

priority_queue Adaptor.

- Lines 17–20 use the **empty** member function (available in all containers) to determine whether the priority_queue is empty (line 17).
 - If not, line 18 uses priority_queue function **top** to retrieve the highest-priority element (i.e., the largest value) in the priority_queue for output.
 - Line 19 calls **pop**, available in all adaptor classes, to remove the priority_queue's highest-priority element.

```
14      std::cout << "Popping from priorities: ";
15
16      // pop element from priority_queue
17      while (!priorities.empty()) {
18          std::cout << priorities.top() << ' '; // view top element
19          priorities.pop(); // remove top element
20      }
21
22      std::cout << "\n";
23  }
```

Popping from priorities: 9.8 5.4 3.2

bitset Near Container

- Class `bitset` (header `<bitset>`) makes it easy to **create and manipulate bit for representing a set of bit flags**.
- bitsets are fixed in size at compile-time.
- Class `bitset` is an alternative tool for bit manipulation.

bitset Near Container (Cont.)

- The following declaration creates bitset `b`, in which every one of the size bits is initially 0 (“off”):
`bitset<size> b;`
- The statement
`b.set(bitNumber);`
sets bit `bitNumber` “on.” You also can call this function with a bool second argument specifying whether to set the bit “on” (true) or “off” (false). The expression `b.set()` sets all bits in `b` “on.”
- The statement
`b.reset(bitNumber);`
sets bit `bitNumber` “off.” The expression `b.reset()` sets all bits in `b` “off.”
- The statement
`b.flip(bitNumber);`
toggles bit `bitNumber`—if the bit is “on,” flip sets it “off,” and vice versa. The expression `b.flip()` flips all bits in `b`.

bitset Near Container (Cont.)

- The expression `b[bitNumber]` for a non-const bitset returns a `std::bitset::reference` that enables you to manipulate the bool at position `bitNumber`; otherwise, it returns a copy of the bool.
- The expression `b.test(bitNumber);` performs range checking on `bitNumber` first.
 - If `bitNumber` is in range (based on the number of bits in the bitset), `test` returns true if the bit is on or false if it's off.
 - Otherwise, `test` throws an `out_of_range` exception.

bitset Near Container (Cont.)

- The expression `b.size()` returns the number of bits.
- The expression `b.count()` returns the number of bits that are set (true).
- The expression `b.any()` returns true if any bit is set.
- The expression `b.all()` returns true if all of the bits are set (true).
- The expression `b.none()` returns true if none of the bits is set (that is, all the bits are false).

bitset Near Container (Cont.)

- The expressions $b == b1$, $b != b1$ compare the two bitsets for equality and inequality, respectively.
- Each of the bitwise assignment operators $\&=$, $|\=$ and $\^{\wedge}=\$ can be used to combine bitsets:
 - Bitwise AND
 $b \&= b1$;
performs a bit-by-bit AND between b and $b1$, setting each bit in b “on” if it’s “on” in both b and $b1$.
 - Bitwise OR
 $b |\= b1$;
sets each bit in b “on” if it’s “on” in either or both of b and $b1$.
 - Bitwise XOR
 $b \^{\wedge}=\ b2$;
sets each bit in b “on” if it’s “on” in only b and $b1$, but not both.
- You also can perform a bitwise NOT operation. The following expression returns a copy of the bitset with all its bits flipped:
 $\sim b$

bitset Near Container.

- The statement
`b >>= n;`
shifts the bits right by `n` positions.
- The expression
`b <<= n;`
shifts the bits left by `n` positions.
- The expressions
`b.to_string()`
`b.to_ulong()`
`b.to_ullong()`
convert the bitset to a string, an unsigned long or an unsigned long long, respectively.